



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA

Laurea Triennale in Informatica L31

Curriculum: Tecnologie Informatiche

Tirocinio

Confronto di tecniche di embeddings in ambienti di Graph Data Science distribuiti: un caso di studio in ambito biomedico

Relatore:

Prof. Antonio Celesti

Candidato:

Sandi Russo

Anno Accademico:

2025/2026

Indice

1	Introduzione	2
1.1	Contesto Operativo	2
1.2	Problematica Affrontata	3
1.3	Obiettivi del lavoro svolto	5
2	Fondamenti di Graph Data Science	6
2.1	I paradigmi di rappresentazione: RDF e Property Graph	6
2.1.1	Il modello Resource Description Framework (RDF)	7
2.1.2	Il modello Property Graph	7
2.2	NoSQL Graph DBMS	8
2.3	Graph Embeddings e Graph Neural Networks	9
2.4	Graph Neural Networks Induttive	11
2.4.1	GraphSAGE	11
2.4.2	HashGNN	14
2.5	Graph Neural Networks Trasduttive	15
2.5.1	TransE	15
2.6	Link Prediction via Graph Embeddings	17
2.6.1	GraphSAGE	18
2.6.2	TransE	19
2.6.3	HashGNN	20

3	Metodologia e Tecnologie	21
3.1	Motivazione della scelta degli algoritmi	22
3.2	Architettura dell'esperimento	23
3.3	Strumenti e tecnologie	24
3.3.1	Neo4j Enterprise	24
3.3.2	Libreria Graph Data Science	25
3.3.3	PyTorch Geometric	26
3.3.4	Containerizzazione e automazione infrastrutturale	26
3.4	Il dataset: PharMeBiNet V3	27
4	Implementazione	28
4.1	Configurazione dell'Ambiente e del Cluster Neo4j	28
4.2	Acquisizione e Ingestion del Dataset	30
4.3	Data Preprocessing ed Estrazione del Sottografo	32
4.4	Implementazione dei Modelli di Embedding	35
4.5	Implementazione del Classificatore Binario (MLP)	37
4.6	Definizione della Pipeline Sperimentale	38
5	Risultati e Discussione	40
5.1	Panoramica dei risultati	40
5.2	Benchmark infrastrutturale	41
5.2.1	Tempi di estrazione al variare dei nodi attivi	41
5.2.2	Analisi dei risultati infrastrutturali	42
5.3	Risultati della Link Prediction	43
5.3.1	Metriche per modello	43
5.3.2	Confronto grafico	44
5.4	Discussione	45
5.4.1	Analisi delle prestazioni di GraphSAGE	45
5.4.2	Analisi delle prestazioni di HashGNN	46
5.4.3	Analisi delle prestazioni di TransE	47

5.4.4	Analisi del trade-off tra accuratezza ed efficienza	48
5.4.5	Verifica delle ipotesi sperimentali	48
6	Conclusioni	50
6.1	Sintesi del lavoro	50
6.2	Discussione dei Risultati	51
6.2.1	Analisi Comparativa delle Architetture Predittive	51
6.2.2	Scalabilità e Overhead Infrastrutturale	51
6.3	Contributi del lavoro	52
6.4	Limitazioni e Sviluppi Futuri	52

Elenco delle figure

2.1	Esempio di Property Graph con nodi etichettati, tipi di relazione e proprietà associate	8
2.2	Processo di generazione di GEs.	10
2.3	Il meccanismo di aggregazione di GraphSAGE	14
2.4	L'intuizione geometrica di TransE: la relazione r è un vettore di traslazione da h a t nello spazio degli embedding.	16
2.5	Pipeline architetturale di GraphSAGE	18
2.6	Pipeline di TransE per la link prediction.	19
2.7	Architettura di HashGNN	20
3.1	Ciclo di vita dell'esecuzione degli algoritmi tramite la libreria Neo4j GDS: il grafo viene proiettato in memoria, l'algoritmo viene eseguito sulla proiezione e i risultati vengono scritti nel database.	25
3.2	Schema del KG PharMeBiNet [1].	27
5.1	Tempi di estrazione nelle tre configurazioni del cluster. La linea tratteggiata evidenzia il baseline a 1 nodo.	42
5.2	AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti. L'asse Y parte da 0.975 per rendere visibili le differenze tra i modelli.	44
5.3	Tempo medio di training per i tre modelli su 5 run indipendenti	45
5.4	Trade-off tra accuratezza ed efficienza computazionale per i tre modelli.	49

Elenco delle tabelle

4.1	Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3. .	33
4.2	Iperparametri condivisi tra i tre modelli.	39
5.1	Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.	41
5.2	Variazione percentuale dei tempi di estrazione rispetto al baseline a 1 nodo.	41
5.3	Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.	44
5.4	Risultati dettagliati per singola run, raggruppati per modello.	44

Listings

4.1	Configurazione memoria Neo4j (neo4j.conf)	29
4.2	Installazione delle librerie della pipeline sperimentale	30
4.3	Caricamento del dump PharMeBiNet V3	31
4.4	Creazione del database distribuito con seeding	31
4.5	Query EDA su PharMeBiNet	32
4.6	Query sul grado dei nodi principali	33
4.7	Query di estrazione del sottografo Chemical→Disease	34
4.8	Costruzione dell’oggetto HeteroData per PyTorch Geometric	34
4.9	Fusione degli spazi di embedding con offset in GraphSAGE	35
4.10	Funzione di scoring di TransE	36
4.11	Funzione di hashing LSH in HashGNN	36
4.12	Suddivisione training/test con RandomLinkSplit	38
4.13	Esecuzione della pipeline completa	39

Sommario

L'efficacia e la scalabilità di tre algoritmi di Graph Machine Learning vengono analizzate applicandoli a un knowledge graph biomedico di grandi dimensioni. È stato scelto il dataset PharMeBiNet V3 [2], una rete eterogenea con circa 8 milioni di nodi distribuiti su oltre 80 tipi e 53 milioni di relazioni distribuite su 410 tipi distinti, che interconnette composti chimici, geni, proteine e patologie provenienti da oltre 40 banche dati farmacologiche e biologiche.

Il problema studiato è la link prediction tra nodi *Chemical* e *Disease*, biologicamente rilevante nel contesto del drug repurposing, ovvero prevedere relazioni terapeutiche non ancora documentate tra farmaci esistenti e malattie note. Il sottografo estratto per gli esperimenti comprende 594.343 nodi Chemical, 23.580 nodi Disease e 123.361 relazioni dirette tra loro. I tre algoritmi confrontati sono GraphSAGE, TransE e HashGNN, scelti per le loro diverse strategie di rappresentazione, rispettivamente: omogenea con aggregazione dei vicini, traslazionale nello spazio vettoriale, ed eterogenea con proiezioni differenziate per tipo di nodo.

L'infrastruttura sperimentale è un cluster Neo4j Enterprise 2026.04.0 distribuito su quattro macchine virtuali cloud, nella configurazione di 1 primary e 3 secondary. Il benchmark di scalabilità valuta l'impatto del numero di nodi attivi sui tempi di estrazione dei dati verso la pipeline PyTorch Geometric, variando la configurazione tra 1, 2 e 4 nodi.

I risultati mostrano che GraphSAGE raggiunge la precisione più alta con un AUC-ROC medio di 0.993 su cinque run indipendenti, mentre TransE si rivela il modello più efficiente, registrando tempi di training circa tre volte inferiori pur mantenendo una qualità predittiva eccellente (AUC-ROC $\geq$ 0.98). Sul fronte infrastrutturale, il cluster distribuito non produce speedup significativi su query sequenziali di estrazione batch, evidenziando come il collo di bottiglia sia legato all'overhead di coordinamento di rete piuttosto che alla disponibilità di risorse computazionali.

Acknowledgement

Desidero esprimere la mia più sincera gratitudine al Prof. Antonio Celesti e al dott. Rosario Napoli per avermi concesso la preziosa opportunità di svolgere il tirocinio sotto la loro supervisione. Li ringrazio per la fiducia accordatami e per avermi permesso di esplorare e approfondire tematiche così stimolanti e all'avanguardia per il mio lavoro di tesi.

Un ringraziamento profondo e speciale va ai miei genitori. Grazie per il vostro sostegno incondizionato, per i sacrifici fatti e per esserci sempre stati al mio fianco, incoraggiandomi a non arrendermi mai nonostante le difficoltà incontrate lungo questo cammino. Se oggi ho raggiunto questo traguardo accademico, è soprattutto merito vostro.

Infine, dedico questo lavoro a mio nonno materno, che ci ha lasciati da oltre undici anni ma il cui ricordo mi accompagna e mi guida ogni giorno. Questa laurea è anche per te, con la speranza che io possa renderti orgoglioso, il tuo 'buffone'.

Capitolo 1

Introduzione

1.1 Contesto Operativo

Negli ultimi decenni, l'analisi dei sistemi complessi ha subito una trasformazione radicale grazie all'evoluzione delle metodologie di estrazione della conoscenza. Se inizialmente l'attenzione era focalizzata sul paradigma dei Big Data, caratterizzato dalla gestione di volumi massicci e flussi veloci di informazioni [3], oggi la vera sfida risiede nella comprensione delle interconnessioni latenti tra i dati.

In questo scenario si inserisce la **Graph Data Science (GDS)**, una disciplina che combina la potenza della *Big Data Analytics* con l'espressività semantica e strutturale dei **Knowledge Graphs (KGs)**, ovvero strutture dati a grafo i cui nodi e relazioni presentano delle *labels* semantiche. La GDS supera i limiti delle rappresentazioni tabulari tradizionali isolando non solo le entità, ma modellandone esplicitamente le relazioni su basi deterministiche, trasformando la topologia della rete stessa in un'informazione direttamente computabile.

In bioinformatica, e in particolare in ambito farmacologico, questo approccio risulta altamente affine. Un singolo farmaco, infatti, può interagire con decine di entità differenti, generando reti di dipendenze che nessun approccio tradizionale riesce a rappresentare in modo efficace.

La ricerca e lo sviluppo di nuovi farmaci richiedono l'integrazione di dati eterogenei, quali: sequenze genomiche, strutture proteiche, interazioni farmaco-target con relativi effetti collaterali documentati, provenienti da fonti diverse e con formati incompatibili tra loro. In questo scenario, il legame tra due entità biologiche (ad esempio, l'inibizione di un enzima da parte di un composto) trasporta un'informazione tanto rilevante quanto le entità stesse, poiché è da quella relazione che si deduce l'effetto farmacologico o la potenziale tossicità. Per questo motivo, lo studio delle reti biologiche non può limitarsi a catalogare le entità coinvolte, ma deve abbracciare il paradigma GDS per modellarne esplicitamente le connessioni.

Emerge quindi un duplice problema: da un lato, la crescita quantitativa dei dati biomedici; dall'altro, la loro natura intrinsecamente interconnessa, che richiede l'applicazione di metodologie in grado di estrarre e analizzare i legami che le connettono.

1.2 Problematica Affrontata

I sistemi biologici sono caratterizzati da un'eterogeneità strutturale tale da rendere insufficiente qualsiasi approccio che tratti le entità in isolamento: una singola molecola può essere coinvolta in decine di interazioni con geni, proteine e patologie, e il significato biologico di ciascuna entità emerge solo se osservato all'interno di questa complessa topologia. Compiti come l'identificazione di potenziali relazioni terapeutiche non ancora documentate tra composti chimici e patologie, alla base del *drug repurposing*, richiedono quindi strumenti di *Graph Machine Learning* (GML) capaci di elaborare non solo le entità coinvolte, ma anche l'intera struttura di connessioni che le lega. Si tratta di un requisito che i tradizionali metodi di analisi dei dati non sono progettati per soddisfare in modo nativo.

Non esiste tuttavia un'unica soluzione algoritmica riconosciuta come ottimale per affrontare questo tipo di compito su Knowledge Graph di grande scala. In particolare, esistono differenti approcci di rappresentazione e manipolazione tra entità e le loro interazioni, con conseguenti compromessi tra qualità predittiva ed onere computazionale.

A questa complessità algoritmica si affianca un problema infrastrutturale: i dati necessari per applicare la GDS a questi domini sono distribuiti su cluster di grandi dimensioni. La loro estrazione verso strumenti di elaborazione esterni (o l'analisi nativa in-database) richiede un'infrastruttura in grado di scalare orizzontalmente, senza che l'overhead di coordinamento tra i nodi del cluster ne comprometta i benefici. Si tratta di un aspetto spesso trascurato nella letteratura a favore della sola accuratezza predittiva dei modelli.

Attualmente, la letteratura scientifica ha esplorato ampiamente l'uso del *Graph Machine Learning* per il drug repurposing e l'estrazione di conoscenza biomedica, consolidando due filoni principali. Da un lato, i metodi basati su embedding geometrici trasduttivi, come **TransE** [4], si sono dimostrati particolarmente efficaci nel preservare la semantica specifica delle singole relazioni. Dall'altro, le architetture induttive basate su *message passing*, come **GraphSAGE** [5], hanno mostrato la capacità di aggregare informazioni topologiche esplorando il vicinato dei nodi e di generalizzare a entità non osservate durante il training. Recentemente, approcci come **HashGNN** [6] hanno introdotto meccanismi per gestire esplicitamente l'eterogeneità del grafo attraverso rappresentazioni differenziate per tipologia di nodo.

Tuttavia, dall'analisi dello stato dell'arte emergono due lacune rilevanti. Dal punto di vista algoritmico, la maggior parte dei lavori esistenti si concentra sulla massimizzazione delle performance predittive su dataset di riferimento spesso pre-processati, omogenei o di dimensioni contenute. Manca un'analisi comparativa strutturata che valuti l'impatto diretto dei diversi assunti architetturali, ovvero la distinzione tra approcci omogenei ed eterogenei e tra paradigmi induttivi e trasduttivi, su reti biomediche reali di scala massiva. In KG di grandi dimensioni, l'elevata densità relazionale e la forte eterogeneità dei tipi di nodo alterano radicalmente il comportamento teorico dei modelli, rendendo insufficienti le valutazioni condotte su reti più semplici.

La quasi totalità della letteratura di settore assume che il grafo risieda già interamente nella memoria centrale dell'ambiente di calcolo, ignorando la fase critica di estrazione e transito dei dati da un sistema di storage distribuito verso il framework di apprendimento.

In contesti reali, la latenza generata dal trasferimento dei dati e l’overhead di coordinamento di un cluster di database possono rappresentare il principale collo di bottiglia dell’intera pipeline analitica. Ciò che manca in letteratura è pertanto una valutazione *end-to-end* che non si limiti a misurare l’efficacia teorica dei paradigmi di embedding, ma che li collochi in un contesto infrastrutturale realistico, quantificando la sostenibilità computazionale del processo di estrazione dell’informazione.

1.3 Obiettivi del lavoro svolto

Il presente lavoro propone un’analisi che si colloca all’intersezione tra GDS ed architetture distribuite, basata sul KG biomedico PharMeBiNet V3 [2], una famosa rete eterogenea di grandi dimensioni, composta da circa 8 milioni di nodi e 53 milioni di relazioni provenienti da molteplici database farmacologici e biologici.

Il task considerato è la *link prediction*, problematica centrale nel dominio del *drug repurposing*, con l’obiettivo di identificare potenziali relazioni terapeutiche non ancora documentate, sfruttando la struttura del KG e le informazioni latenti codificate nelle sue connessioni. L’analisi si concentra sul confronto tra tre algoritmi: GraphSAGE [5], TransE [4] e HashGNN [6]. Questi modelli rappresentano tre paradigmi distinti di apprendimento su grafi: l’aggregazione di informazioni nei vicini (GraphSAGE), la modellazione geometrica delle relazioni nello spazio latente (TransE) e la gestione esplicita dell’eterogeneità tramite rappresentazioni differenziate per tipo di nodo (HashGNN).

Dal punto di vista infrastrutturale, il lavoro prevede l’uso di un Graph DBMS distribuito su quattro macchine virtuali, configurato secondo un’architettura primary-secondary. L’obiettivo è analizzare l’impatto della scalabilità orizzontale sui tempi di estrazione dei dati necessari all’addestramento nello spazio latente, variando il numero di nodi attivi nel cluster.

In questo contesto, la valutazione non si limita all’accuratezza predittiva dei modelli, ma estende l’analisi alle prestazioni end-to-end della pipeline per quantificare il reale contributo dell’infrastruttura distribuita in complessi scenari di GDS su larga scala.

Capitolo 2

Fondamenti di Graph Data Science

L'adozione della GDS come paradigma per l'analisi di sistemi complessi richiede una modellazione nativamente incentrata sulle relazioni. Per tale ragione, è necessario definire i modelli logici e le strutture dati capaci di ospitare ed elaborare la conoscenza deterministica espressa dai KGs.

Questo capitolo definisce le fondamenta teoriche su cui si sviluppa l'intero lavoro sperimentale. Viene esaminata la tassonomia dei modelli di rappresentazione dei KGs, introducendo il dualismo tra i paradigmi *Resource Description Framework* (RDF) e *Property Graph*. Infine, ci si sposta sui paradigmi di GML, approfondendo le basi metodologiche delle architetture di *Graph Embedding* e delle *Graph Neural Networks* (GNN) per task predittivi, con particolare attenzione alla distinzione tra approcci induttivi e trasduttivi.

2.1 I paradigmi di rappresentazione: RDF e Property Graph

La modellazione formale dei KGs si è storicamente consolidata attorno a due paradigmi principali che rispondono a requisiti e task strutturalmente differenti: il modello **RDF** (*Resource Description Framework*) e il modello **Property Graph** (o *Labeled Property Graph*, LPG).

2.1.1 Il modello Resource Description Framework (RDF)

Il modello **RDF**, sviluppato dal W3C, nasce nell'alveo del Semantic Web con l'obiettivo primario di garantire l'interoperabilità globale dei dati e lo scambio di conoscenza standardizzato sul web. In RDF, l'informazione viene scomposta in triple atomiche formali composte da *Soggetto*, *Predicato* e *Oggetto*, in cui ogni risorsa è identificata univocamente tramite un URI/IRI globale. Questo approccio è strettamente legato a logiche formali e ontologie strutturate (tramite RDFS e OWL) che abilitano motori di inferenza deterministici capaci di derivare nuova conoscenza implicita a partire dalle triple dichiarate. Tuttavia, l'estrema granularità del modello, che richiede la reificazione delle relazioni per poter assegnare proprietà agli archi, e l'onere computazionale del ragionamento ontologico globale rendono RDF meno efficiente quando l'obiettivo si sposta verso l'analisi topologica intensiva e il calcolo distribuito.

2.1.2 Il modello Property Graph

Il modello **Property Graph**, noto anche come *Labeled Property Graph* (LPG), è orientato alle prestazioni, alla scalabilità ed all'attraversamento efficiente di reti fitte (*graph traversal*). Invece di frammentare la conoscenza in triple atomiche, l'LPG consente di arricchire nativamente sia i nodi sia gli archi con un insieme flessibile di proprietà espresse come coppie chiave-valore. Questo paradigma mira all'ottimizzazione dell'efficienza computazionale all'interno di un dominio applicativo circoscritto.

Un property graph è un multigrafo orientato composto da due elementi fondamentali:

- I **nodi** (o vertici): rappresentano le entità. Sono caratterizzati da un identificatore univoco, un insieme di *label* che ne classificano la natura semantica e una collezione di proprietà espresse come coppie chiave-valore. Le label consentono di raggruppare i nodi in insiemi omogenei e possono essere modificate dinamicamente senza alterare lo schema del grafo. La possibilità di assegnare zero o più label simultaneamente permette di modellare entità appartenenti a categorie multiple, senza ricorrere a tabelle di associazione aggiuntive.

- Gli **archi** (o relazioni): descrivono le connessioni tra i nodi. Ogni arco possiede un identificatore univoco, un nodo sorgente e un nodo destinazione, un’etichetta che ne definisce il tipo semantico e un’eventuale collezione di proprietà. A differenza dei nodi, ogni arco deve avere un tipo e una direzione esplicita, sebbene quest’ultima possa essere ignorata dal motore di calcolo durante le query che richiedono un attraversamento non orientato.

Questa flessibilità strutturale si adatta perfettamente alla modellazione di domini complessi.

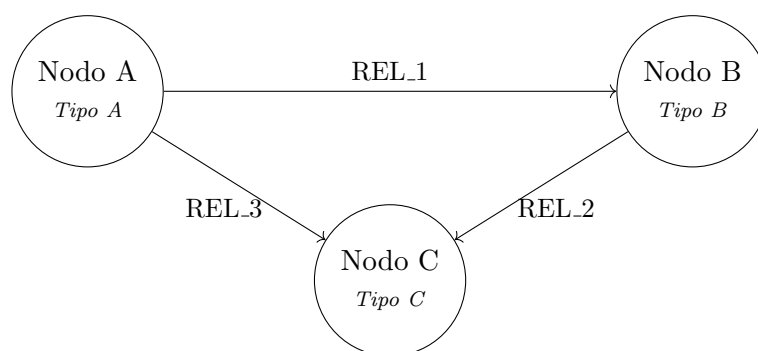


Figura 2.1: Esempio di Property Graph con nodi etichettati, tipi di relazione e proprietà associate

2.2 NoSQL Graph DBMS

La traduzione del modello logico del *Property Graph* in strutture efficienti ha guidato la nascita e l’affermazione dei **Graph DBMS** (Graph Database Management Systems). Questi sistemi si collocano all’interno del più ampio paradigma dei database **NoSQL** (“Not Only SQL”), un movimento tecnologico emerso per superare la rigidità strutturale, i limiti di scalabilità verticale e le inefficienze computazionali dei tradizionali sistemi relazionali (RDBMS) di fronte a carichi di lavoro non convenzionali. L’obiettivo primario non è la memorizzazione di entità disconnesse, ma l’ottimizzazione delle query di attraversamento (*graph traversal*) su strutture dati massivamente interconnesse.

In un Graph DBMS, ogni record associato a un nodo memorizza direttamente nei propri blocchi di memoria i puntatori fisici agli archi incidenti, i quali a loro volta contengono i riferimenti immediati ai nodi adiacenti. Di conseguenza, una volta identificato il nodo di partenza, l'accesso ai suoi vicini diretti avviene tramite una dereferenziazione diretta dei puntatori. Il costo di ogni singolo passo di traversata si riduce a un tempo costante $O(1)$, rendendo la performance della query totalmente indipendente dalla dimensione complessiva del database (n) e legata esclusivamente alla densità locale della porzione di grafo esplorata. Questa architettura rende i Graph DBMS lo strumento d'elezione per contesti ad alta densità relazionale, come la bioinformatica e la farmacologia computazionale.

2.3 Graph Embeddings e Graph Neural Networks

Nella GDS i tasks predittivi necessitano di una transizione dal dominio simbolico-discreto del *Graph DBMS* ad uno spazio continuo-matematico. Questo processo consente di condensare la complessità topologica e semantica di un KG in strutture geometriche dense e di piccole dimensioni, rendendo l'informazione direttamente fruibile dai moderni modelli di apprendimento automatico.

In questo contesto, i **graph embedding** si configurano come funzioni di mappatura volte a proiettare i nodi di una rete in uno spazio vettoriale continuo a bassa dimensionalità. L'obiettivo di questa trasformazione è convertire le entità discrete della rete in vettori numerici, preservando al contempo le proprietà strutturali e le relazioni semantiche del grafo originale. In questo modo, elementi topologicamente vicini o funzionalmente simili nel sistema complesso tendono a convergere in coordinate geometricamente prossime all'interno dello spazio latente di arrivo (Fig. 2.2).

Per superare la rigidità dei metodi di *Deep Learning* applicato a domini non euclidei, si sono affermate le **Graph Neural Networks** (GNN). A differenza degli approcci classici, le GNN non si limitano ad assegnare un vettore statico a ciascuna entità, ma apprendono le rappresentazioni in modo dinamico e induttivo.

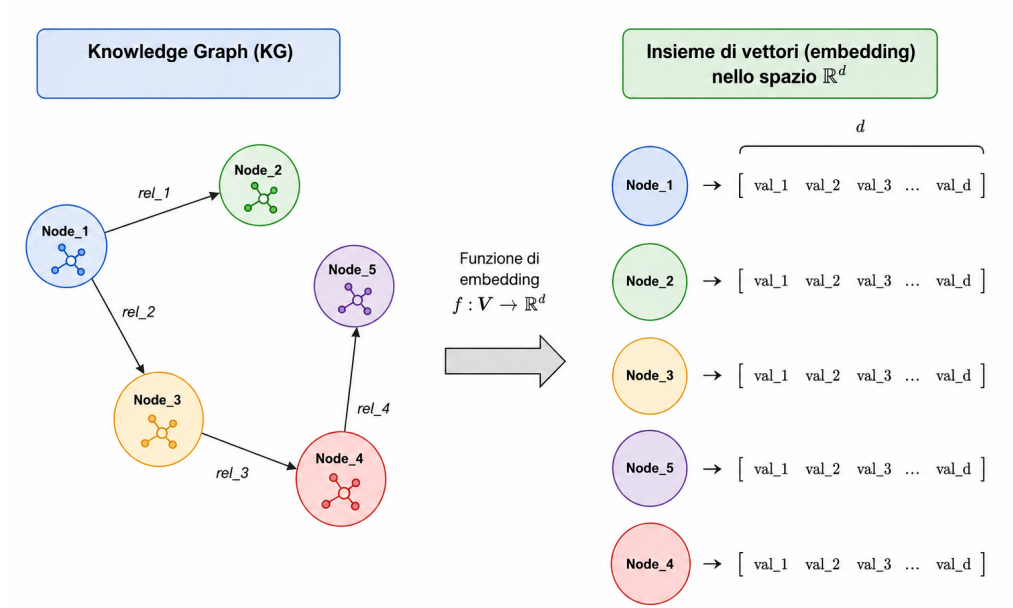


Figura 2.2: Processo di generazione di GEs.

Il nucleo che accomuna gran parte di queste architetture è il paradigma del **message passing**: iterativamente, l'embedding di ciascun nodo viene aggiornato aggregando le rappresentazioni vettoriali dei suoi vicini diretti. Ripetendo questo processo per k iterazioni (o layer), l'embedding di un nodo arriva a catturare le dipendenze strutturali e semantiche del suo intero vicinato fino a k salti di distanza nella rete.

Formalmente, dato un nodo v al layer l , la transizione di stato segue lo schema:

$$\mathbf{h}_v^{(l)} = \text{UPDATE}(\mathbf{h}_v^{(l-1)}, \text{AGGREGATE}(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\})) \quad (2.1)$$

dove $\mathcal{N}(v)$ indica l'insieme dei vicini di v .

Le funzioni AGGREGATE e UPDATE definiscono l'identità dell'architettura: la prima deve essere strettamente invariante rispetto alle permutazioni (poiché l'ordine con cui si leggono i nodi vicini in un grafo non ha rilevanza), mentre la seconda si occupa di combinare l'informazione appena aggregata con lo stato precedente del nodo stesso tramite trasformazioni non lineari.

Le architetture di GML si dividono in approcci **trasduttivi** e quelli **induttivi**.

Un modello **induttivo** apprende una *funzione* di aggregazione generalizzabile piuttosto che vettori fissi per singole entità. Questa funzione può essere applicata a qualsiasi nodo del grafo, inclusi quelli non osservati durante il training, producendo un embedding a partire esclusivamente dalla struttura locale del suo vicinato e dalle feature dei nodi circostanti. L'approccio induttivo è particolarmente rilevante in domini dinamici, dove nuove entità vengono continuamente integrate in reti preesistenti, rendendo impraticabile il riaddestramento sistematico del modello.

Un modello **trasduttivo** ottimizza una rappresentazione vettoriale specifica per ciascun nodo osservato durante il training: l'embedding è associato all'identità del nodo, non alla sua posizione strutturale nella rete. Questa caratteristica implica che il modello non possa generalizzare a nodi non visti: in presenza di nuove entità aggiunte al grafo dopo il training, l'intero processo di apprendimento deve essere ripetuto da capo.

2.4 Graph Neural Networks Induttive

2.4.1 GraphSAGE

GraphSAGE (*Graph SAmple and aggreGatE*) è un framework per l'apprendimento di graph embedding induttivi, introdotto formalmente da Hamilton et al. nel 2017 [5]. A differenza degli approcci trasduttivi che ottimizzano una matrice di embedding fissa esclusivamente per i nodi osservati durante la fase di training, GraphSAGE apprende una **funzione di aggregazione** topologica generalizzabile. L'idea alla base di GraphSAGE è che l'identità semantica e strutturale di un nodo possa essere inferita dalla composizione del suo vicinato. Invece di addestrare un vettore individuale per ciascuna entità, il modello apprende un set di matrici di peso che combinano le feature dei nodi adiacenti, permettendo di generalizzare a nodi mai visti durante il training. Questo approccio permette di generare embedding *on-the-fly* per nodi completamente invisibili durante l'addestramento, rendendo l'algoritmo intrinsecamente adatto a grafi dinamici e reti in continua evoluzione.

GraphSAGE, quindi, può produrre embedding per un nuovo nodo semplicemente osservando il suo vicinato nel grafo aggiornato, senza alcun riaddestramento. Una volta completato il training, il modello addestrato può essere inserito direttamente in una pipeline di inferenza e applicato a nodi non visti, purché questi possiedano un vettore di feature della stessa dimensione e con lo stesso significato semantico usato durante l'addestramento, poiché le operazioni di aggregazione (media, somma) vengono eseguite tra vettori che devono essere comparabili nello stesso spazio.

GraphSAGE però **non tiene conto delle label dei nodi né dei tipi di relazione**: aggrega i vicini indiscriminatamente, trattando il grafo come se fosse omogeneo. Questo lo rende meno adatto a catturare la ricchezza semantica di grafi fortemente eterogenei, dove tipi di relazione distinti descrivono interazioni con significati molto diversi tra loro. Per superare il problema dell'*esplosione del vicinato* (i.e., il calcolo esponenziale generato dalle funzioni di aggregazione), il *forward pass* di GraphSAGE introduce un meccanismo di campionamento stocastico. In particolare, per ogni layer l , l'elaborazione si articola in due fasi:

1. **Campionamento**: per ogni nodo v , viene selezionato uniformemente un sottoinsieme di dimensione fissa $\mathcal{S}(v) \subseteq \mathcal{N}(v)$ di vicini. La dimensione del campione è un iperparametro che controlla il trade-off tra qualità dell'aggregazione e costo computazionale. Questo vincolo rende il costo per singolo nodo costante e prevedibile, indipendentemente dalla densità locale della rete. Nel caso in cui un nodo sia isolato, ovvero privo di vicini, l'algoritmo recupera informazioni esclusivamente dallo stato corrente del nodo stesso durante lo step di aggregazione.
2. **Aggregazione**: le rappresentazioni dei nodi campionati vengono aggregate in un singolo vettore di contesto, il quale riassume l'informazione proveniente dal vicinato locale. Questo vettore viene poi fuso con la rappresentazione precedente del nodo target tramite concatenazione e trasformazione lineare, così da produrre un nuovo embedding che integra sia le informazioni strutturali sia quelle già codificate al layer precedente.

Formalmente, le equazioni di aggiornamento al layer l sono definite come:

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \text{AGGREGATE}_l (\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{S}(v)\}) \quad (2.2)$$

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \left[\mathbf{h}_v^{(l-1)} \parallel \mathbf{h}_{\mathcal{N}(v)}^{(l)} \right] \right) \quad (2.3)$$

dove $\parallel$ indica l'operazione di concatenazione vettoriale, $\mathbf{W}^{(l)}$ rappresenta la matrice dei pesi ottimizzabile al layer l , e σ denota una funzione di attivazione non lineare. La scelta della concatenazione rispetto alla semplice somma è cruciale: preserva la distinzione tra lo stato storico del nodo e l'informazione fresca derivante dal vicinato, una distinzione che la somma cancellerebbe collassando i due contributi in un unico vettore indistinguibile. Le funzioni di attivazione supportate includono ReLU, Leaky ReLU e Sigmoid, con ReLU come scelta standard per i layer intermedi.

Con k layer impilati, ogni nodo aggrega informazioni strutturali fino a k salti di distanza nel grafo. Con un solo layer, ogni nodo conosce solo i suoi vicini diretti. Con due layer, la configurazione adottata in questo lavoro, ogni nodo integra informazioni sul vicinato del vicinato, catturando pattern strutturali locali di secondo ordine. Aumentare la profondità oltre 2-3 layer introduce il fenomeno dell'**over-smoothing**: le rappresentazioni dei nodi tendono a convergere verso valori simili, perdendo la specificità locale che rende gli embedding utili per task discriminativi. Va inoltre notato che la dimensione degli embedding è direttamente proporzionale alla quantità di informazione catturata: embedding di dimensione maggiore permettono di rappresentare pattern più complessi, ma a costo di un maggiore consumo di memoria.

Affinché l'operazione sia matematicamente robusta, la funzione AGGREGATE deve garantire l'invarianza alle permutazioni. Le implementazioni standard includono l'aggregatore *Mean* (media element-wise dei vettori dei vicini, computazionalmente efficiente), l'aggregatore *Pool* (applicazione di una rete feed-forward a ciascun vicino seguita da max-pooling element-wise, computazionalmente più oneroso) e l'aggregatore *LSTM*, adattato per gestire la natura non ordinata dei vicinati tramite permutazioni casuali.

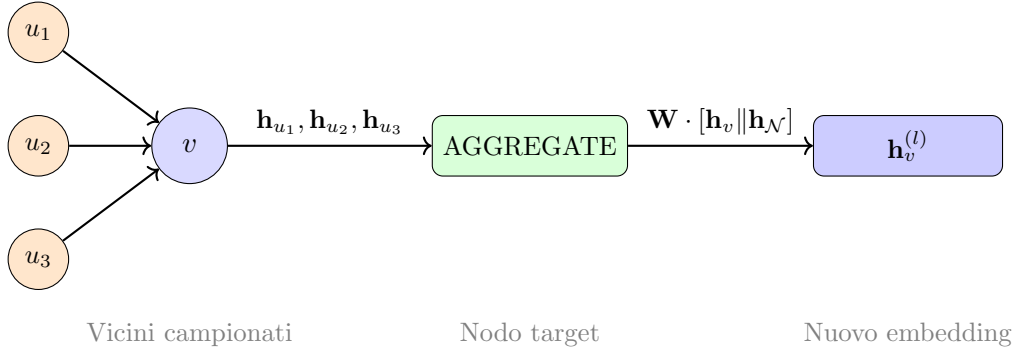


Figura 2.3: Il meccanismo di aggregazione di GraphSAGE

2.4.2 HashGNN

HashGNN è un algoritmo di graph embedding progettato nativamente per grafi eterogenei [6]. A differenza di GraphSAGE, che fonde i diversi tipi di nodo in un unico spazio omogeneo, HashGNN mantiene **spazi di rappresentazione separati** per ogni tipo di nodo, usando funzioni di hashing casuali per arricchire le feature senza aumentare il numero di parametri apprendibili. Questo approccio consente di preservare le specificità semantiche di ciascuna categoria di entità, evitando che caratteristiche eterogenee vengano diluite in una rappresentazione comune.

La motivazione è diretta: in un KG eterogeneo, nodi di tipo diverso rappresentano entità semanticamente incomparabili. Forzarli nello stesso spazio vettoriale fin dall'inizio, come fa GraphSAGE, equivale a proiettare su una mappa comune due lingue diverse senza traduzione. HashGNN evita questo collasso semantico mantenendo coordinate separate per ogni tipo di nodo fino al momento della predizione, quando il decoder MLP apprende la mappatura tra i diversi spazi di rappresentazione.

L'idea centrale di HashGNN è sostituire le convenzionali proiezioni lineari apprendibili con **proiezioni casuali fisse** ispirate al *Locality-Sensitive Hashing* (LSH) [7], una tecnica di hashing probabilistico. L'LSH è una famiglia di tecniche che proiettano dati ad alta dimensionalità in spazi a bassa dimensionalità preservando le distanze fondamentali di input: punti vicini nello spazio originale rimangono vicini anche dopo la proiezione con alta probabilità statistica.

Per ogni nodo, l’embedding viene proiettato attraverso K matrici casuali fisse appositamente generate $\mathbf{R}_1, \dots, \mathbf{R}_K$, e i risultati vengono binarizzati tramite la funzione segno:

$$\mathbf{h}_v^{hash} = [\text{sign}(\mathbf{x}_v \mathbf{R}_1^\top) \parallel \dots \parallel \text{sign}(\mathbf{x}_v \mathbf{R}_K^\top)] \quad (2.4)$$

Questo produce K viste binarie diverse dello stesso nodo, ciascuna che cattura aspetti complementari della struttura locale. La binarizzazione tramite sign è computazionalmente economica e produce rappresentazioni sparse, riducendo il costo delle operazioni successive.

Le proiezioni fisse non vengono aggiornate durante il training: solo i layer lineari successivi sono apprendibili, uno per ogni tipo di nodo. Questo disaccoppiamento tra la fase di hashing (fissa, non ottimizzabile) e la fase di proiezione (apprendibile) riduce il numero totale di parametri e limita il rischio di overfitting.

HashGNN gestisce nativamente l’eterogeneità del grafo mantenendo proiezioni distinte per ogni tipo di nodo. Le proiezioni fisse riducono il numero di parametri apprendibili e il rischio di overfitting, e non richiedono backpropagation attraverso la fase di hashing.

Il principale svantaggio è la **convergenza lenta**: i layer lineari devono compensare la casualità e la binarizzazione delle proiezioni hash, richiedendo più epoche per stabilizzarsi rispetto a GraphSAGE, pur producendo un’accuratezza statisticamente comparabile.

2.5 Graph Neural Networks Trasduttive

2.5.1 TransE

TransE (*Translating Embeddings*) è un modello di *knowledge graph embedding* proposto da Bordes et al. nel 2013 [4]. A differenza delle GNN, che aggregano informazione strutturale dai vicini tramite meccanismi di message passing, TransE apprende embedding per entità e relazioni modellando queste ultime come **traslazioni nello spazio vettoriale**.

Non vi è alcuna aggregazione di vicinato; grafo viene rappresentato come un insieme di triple (h, r, t) , testa, relazione e coda, e il modello impara a posizionare le entità nello spazio latente in modo che la relazione r corrisponda, idealmente, a un vettore di spostamento da h a t . Questo paradigma lo rende particolarmente adatto a KGs con molti tipi di relazione semanticamente distinte.

L'idea di TransE è geometrica: data una tripla valida (h, r, t) , dove h è il nodo testa, r la relazione e t il nodo coda, il modello ottimizza gli embedding affinché $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$. In altre parole, la relazione è un vettore che, sommato all'embedding della testa, deve avvicinarsi il più possibile all'embedding della coda.

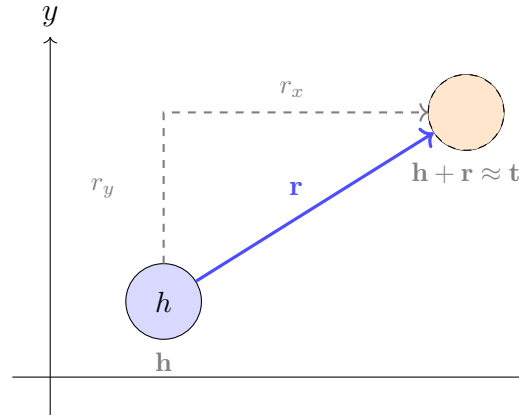


Figura 2.4: L'intuizione geometrica di TransE: la relazione r è un vettore di traslazione da h a t nello spazio degli embedding.

La funzione di scoring per una tripla (h, r, t) è la distanza L2 tra $\mathbf{h} + \mathbf{r}$ e $\mathbf{t}$:

$$f(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2 \quad (2.5)$$

Un valore basso indica una tripla plausibile, mentre un valore alto indica una tripla improbabile. Il training utilizza una **margin ranking loss**:

$$\mathcal{L} = \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} \max(0, \gamma + f(h, r, t) - f(h', r, t')) \quad (2.6)$$

dove S è l'insieme delle triple positive, S' quello delle triple negative generate per corruzione, e γ è il margine. La loss penalizza i casi in cui la tripla negativa non risulta almeno di γ peggiore della tripla positiva. Durante il training, gli embedding vengono inoltre vincolati o normalizzati per evitare che il modello riduca artificialmente la loss aumentando le norme dei vettori.

Gli embedding vengono normalizzati L2 prima di ogni aggiornamento, per evitare che il modello raggiunga loss bassa semplicemente espandendo le norme dei vettori invece di apprendere una struttura geometrica significativa.

TransE è computazionalmente efficiente: non richiede aggregazione di vicinato né propagazione di messaggi. Cattura esplicitamente la semantica delle relazioni tramite il vettore $\mathbf{r}$, il che lo rende teoricamente adatto a knowledge graph con molti tipi di relazione semanticamente distinti. Infine, essendo un modello **trasduttivo** non generalizza a nodi non visti durante il training. Presenta inoltre difficoltà nel modellare relazioni simmetriche ($h \rightarrow t$ e $t \rightarrow h$ dello stesso tipo) poiché la traslazione è direzionale per definizione, e non cattura informazioni strutturali di ordine superiore.

2.6 Link Prediction via Graph Embeddings

La risoluzione del task di link prediction attraverso l'uso dei graph embedding si fonda su un paradigma architetturale di tipo Encoder-Decoder. Indipendentemente dall'algoritmo specifico adottato, il flusso di elaborazione prevede due fasi sequenziali. Nella prima fase, l'Encoder elabora il sottografo in input mappando le entità del dominio discreto in uno spazio vettoriale continuo a bassa dimensionalità, generando così le rappresentazioni latenti che ne codificano le proprietà strutturali e semantiche. Nella seconda fase, il Decoder riceve in input gli embedding di una coppia di nodi candidati e, tramite una funzione di scoring geometrica o una rete neurale dedicata, inferisce uno score continuo che quantifica la probabilità di una connessione reale tra di essi. Nelle sottosezioni seguenti vengono declinate le specifiche pipeline di elaborazione implementate per i tre modelli in esame.

2.6.1 GraphSAGE

La pipeline si articola in tre componenti distinti. L'applicazione del framework induttivo GraphSAGE per la risoluzione del task di link prediction viene declinata attraverso un'architettura dedicata, la cui pipeline si articola in tre componenti distinti.

Encoder: poiché i nodi del grafo non possiedono feature numeriche ma sono identificati solo da stringhe testuali, le feature iniziali sono **embedding learnable**, una tabella di vettori inizializzati casualmente e ottimizzati durante il training. Gli embedding dei due tipi di nodo vengono fusi in un unico tensore omogeneo tramite concatenazione, e gli indici del secondo tipo vengono traslati di `num_nodi_tipo_A` posizioni per evitare collisioni. Due layer SAGEConv propagano l'informazione strutturale del vicinato.

Decoder: dati gli embedding $\mathbf{z}_A$ e $\mathbf{z}_B$ di una coppia di nodi, il decoder li concatena e li passa attraverso un MLP (*Multi-Layer Perceptron*) con attivazione finale sigmoid, producendo uno score $p \in [0, 1]$ che rappresenta la probabilità che il link esista.

Training: il modello viene addestrato su link positivi (archi reali) e link negativi generati casualmente, minimizzando la *binary cross-entropy loss*. Il 10% degli archi viene tenuto nascosto per la valutazione finale tramite `RandomLinkSplit`.

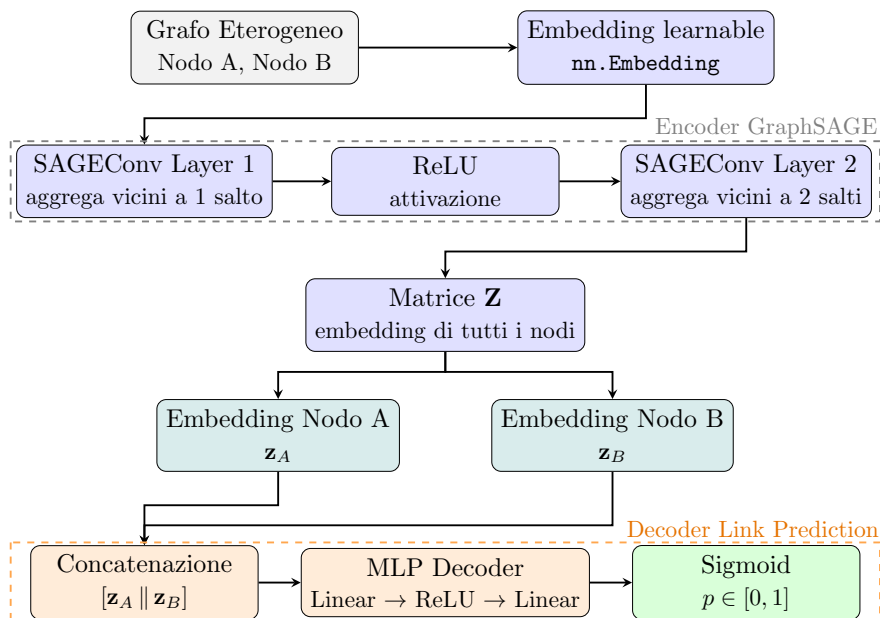


Figura 2.5: Pipeline architetturale di GraphSAGE

2.6.2 TransE

La pipeline di TransE è più semplice di quella di GraphSAGE perché non richiede aggregazione di vicinato: ogni entità e ogni tipo di relazione ha il proprio embedding, e la predizione si basa esclusivamente sulla geometria dello spazio latente.

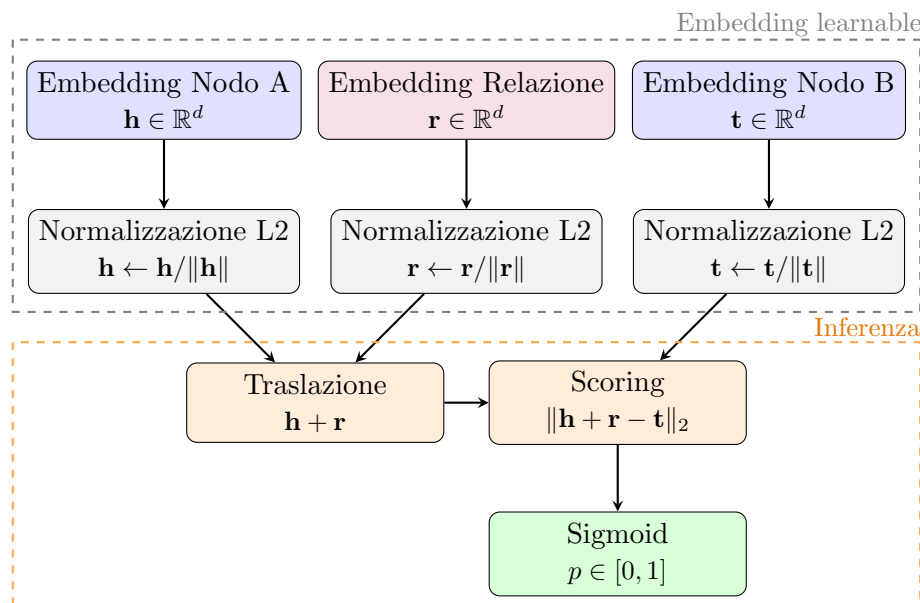


Figura 2.6: Pipeline di TransE per la link prediction.

Embedding: ogni nodo di tipo A, ogni nodo di tipo B e ogni tipo di relazione ha un vettore dedicato, inizializzato con distribuzione uniforme in $[-1, 1]$. Non esiste aggregazione di vicinato: ogni entità impara la propria posizione nello spazio latente indipendentemente dalla struttura locale del grafo.

Scoring e inferenza: data una coppia di nodi, il punteggio viene calcolato come $-\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2$. La sigmoid trasforma il punteggio in una probabilità tra 0 e 1.

Training: la margin ranking loss confronta triple positive con triple negative generate per corruzione casuale, sostituendo la testa o la coda con un'entità scelta a caso. Il margine γ forza una separazione minima tra lo score delle triple reali e di quelle false.

2.6.3 HashGNN

A differenza dell'approccio omogeneo di GraphSAGE, HashGNN affronta esplicitamente la natura multimodale della rete. Anche in questo caso, l'architettura per la previsione dei collegamenti segue la struttura encoder-decoder, articolandosi come segue.

Encoder: gli embedding learnable dei due tipi di nodo vengono processati separatamente attraverso le proiezioni hash e i layer lineari dedicati, producendo due matrici distinte $\mathbf{Z}_A$ e $\mathbf{Z}_B$. I due spazi non vengono mai fusi durante l'encoding, e questa è la differenza architetturale fondamentale rispetto a GraphSAGE.

Decoder: dati gli embedding $\mathbf{z}_A$ e $\mathbf{z}_B$ di una coppia di nodi, il decoder li concatena e li passa attraverso un MLP con attivazione finale sigmoid. È solo in questo punto che le due rappresentazioni si incontrano, e il MLP deve apprendere la regola di compatibilità tra i due spazi eterogenei.

Training: identico a GraphSAGE, binary cross-entropy su link positivi e negativi generati casualmente, con `RandomLinkSplit` per il test set.

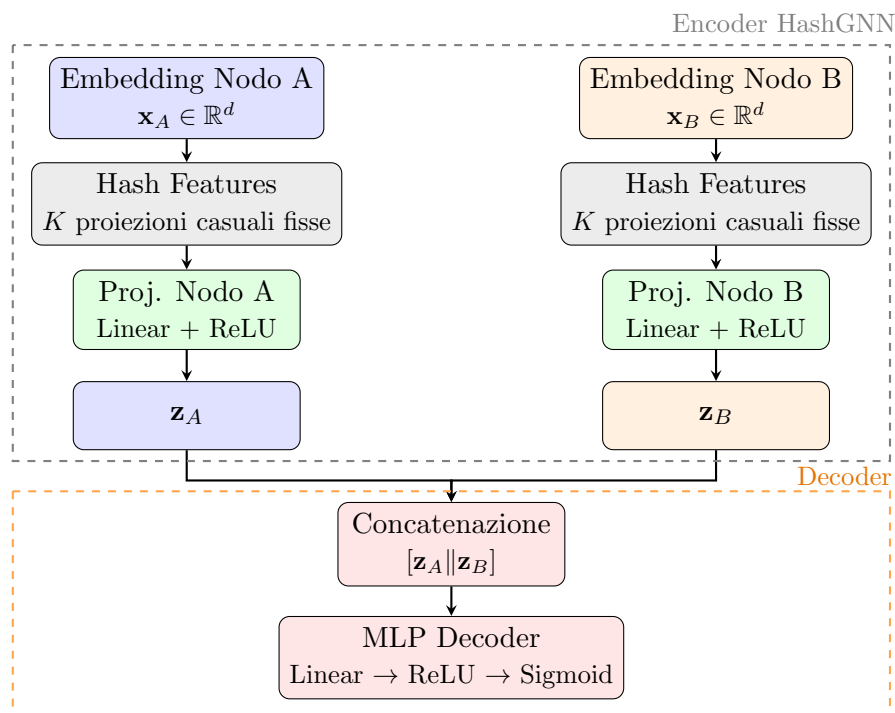


Figura 2.7: Architettura di HashGNN

Capitolo 3

Metodologia e Tecnologie

La GDS offre oggi un ampio catalogo di algoritmi per la generazione di graph embedding, ciascuno fondato su architetture profondamente diverse. Questa varietà, pur costituendo una ricchezza dal punto di vista teorico, pone un problema pratico immediato a chi intende applicare la GDS a un dominio reale: la scelta dell'algoritmo non è banale, poichè ogni modello elabora le informazioni contenute nel KG attraverso le funzioni di aggregazione e produce rappresentazioni vettoriali molto differenti. Di conseguenza, la qualità degli embedding ottenuti, e la loro utilità per un task predittivo specifico, dipende in modo diretto da questa scelta architeturale.

Alla luce di queste considerazioni teoriche, il presente studio si propone di affrontare il problema della selezione algoritmica attraverso una rigorosa valutazione empirica. L'obiettivo centrale consiste nel confrontare diversi approcci di generazione di graph embedding per valutarne oggettivamente la qualità sulle strutture relazionali tipiche di un KG biomedico. Il task di *link prediction* viene adottato come strumento di misurazione diretta: la capacità di un modello di prevedere connessioni non osservate tra entità del grafo riflette direttamente quanto gli embedding prodotti abbiano saputo catturare le proprietà topologiche e semantiche della rete sottostante. In ambito farmacologico, questo corrisponde all'identificazione di relazioni terapeutiche latenti tra entità biologiche, un obiettivo con implicazioni dirette nel dominio del drug repurposing.

3.1 Motivazione della scelta degli algoritmi

La metodologia sperimentale adottata in questo studio si discosta dai tradizionali approcci che si limitano a misurare l'accuratezza predittiva. Al contrario, il presente lavoro propone un'indagine empirica strutturata su un duplice livello di analisi volto a coniugare la valutazione prestazionale dei modelli di apprendimento automatico con l'analisi degli algoritmi di generazione di graph embeddings sottostanti.

L'ipotesi fondante di questa metodologia è che, in scenari applicativi reali come il *drug repurposing*, il valore di una pipeline non dipenda esclusivamente dalla sua capacità di generalizzazione statistica, ma anche dalla sua sostenibilità in scenari caratterizzati da un grande volume. Per condurre una valutazione esaustiva e tracciare un profilo completo delle soluzioni analizzate, l'indagine si sviluppa lungo tre direttrici principali:

- **Valutazione Algoritmica:** analisi della qualità degli *embedding* generati. Il confronto tra approcci induttivi, trasduttivi ed eterogenei ha lo scopo di determinare quale architettura modelli in modo più efficace la struttura e la semantica del grafo biomedico. Le prestazioni vengono misurate in base a tre parametri: l'accuratezza predittiva sulla *link prediction* (calcolata tramite la metrica AUC-ROC), il costo computazionale (tempi di addestramento e velocità di convergenza) e la gestione nativa dell'eterogeneità dei nodi.
- **Valutazione Infrastrutturale:** misurazione delle prestazioni del database distribuito in fase di lettura. Poiché i dati devono essere trasferiti dal layer di *storage* alla pipeline di *Machine Learning*, si analizza il tempo di estrazione sequenziale del sottografo al variare del numero di nodi attivi nel cluster. L'obiettivo è quantificare l'impatto dell'*overhead* di rete e verificare se la scalabilità orizzontale produca un reale aumento del *throughput* o se introduca colli di bottiglia.
- **Validazione Statistica e Riproducibilità:** per garantire che i risultati non siano alterati dall'inizializzazione casuale dei pesi o da fluttuazioni statistiche, ogni esperimento viene eseguito su cinque *run* indipendenti.

L'integrazione di queste tre direttrici consente non solo di decretare quale algoritmo performi meglio in termini assoluti, ma di definire il miglior *trade-off* tra complessità computazionale, accuratezza predittiva e agilità infrastrutturale. La selezione di GraphSAGE, TransE e HashGNN permette di mettere a confronto tre metodi di calcolo profondamente diversi, con l'obiettivo di capire quale di queste strutture sia più efficace nel produrre embedding di alta qualità, in particolare:

- **GraphSAGE** [5] è stato scelto per valutare l'efficacia di un approccio induttivo, verificando quanto la semplice aggregazione omogenea del vicinato locale sia informativa per prevedere nuovi collegamenti;
- **TransE** [4] offre invece una prospettiva opposta e trasduttiva, permettendo di misurare quanto la modellazione puramente geometrica delle relazioni incida sull'accuratezza e fornendo un solido termine di paragone per l'efficienza computazionale;
- **HashGNN** [6], infine, è stato inserito per colmare i limiti di GraphSAGE nella gestione dell'eterogeneità, consentendo di verificare empiricamente se mantenere spazi vettoriali separati per entità differenti produca un reale vantaggio predittivo.

3.2 Architettura dell'esperimento

A partire dal KG di riferimento, viene estratto un sottografo tematico che definisce il perimetro sperimentale, selezionando i tipi di nodo e il tipo di relazione di interesse. Su tale sottografo vengono applicati i tre algoritmi, ciascuno dei quali produce embedding vettoriali per i nodi della rete. La qualità di queste rappresentazioni latenti viene valutata tramite un **classificatore binario** addestrato sul task di *link prediction*: data una coppia di nodi, il modello inferisce la probabilità che tra loro esista una connessione reale, producendo uno score $p \in [0, 1]$.

Questa formulazione permette di misurare direttamente quanto ciascun approccio riesca a catturare la struttura relazionale della rete e a generalizzare su connessioni non osservate durante il training.

In parallelo, l'analisi si estende al livello infrastrutturale, misurando l'impatto della scalabilità orizzontale del sistema di storage distribuito sui tempi di estrazione dei dati verso la pipeline di apprendimento. Questa dimensione è spesso trascurata nella letteratura in favore della sola accuratezza predittiva, ma risulta critica in scenari di elaborazione reale: un overhead di comunicazione e coordinamento eccessivo tra i nodi del cluster può trasformarsi nel principale collo di bottiglia dell'intera pipeline, annullando i benefici computazionali dell'architettura distribuita. Quantificare empiricamente questo impatto, variando il numero di nodi attivi nel cluster, costituisce uno degli obiettivi originali di questo lavoro.

Per assicurare l'affidabilità statistica del confronto, l'intero ciclo di esperimenti viene ripetuto su cinque esecuzioni indipendenti con inizializzazioni diverse, permettendo di distinguere risultati sistematici da fluttuazioni casuali legate all'inizializzazione dei pesi.

3.3 Strumenti e tecnologie

L'implementazione della pipeline sperimentale si basa su un insieme di strumenti complementari, ciascuno selezionato per soddisfare requisiti specifici del flusso di elaborazione.

3.3.1 Neo4j Enterprise

Neo4j è il Graph DBMS adottato per la gestione del Knowledge Graph PharMeBiNet. La sua architettura nativa a Property Graph garantisce attraversamenti in tempo costante $O(1)$ per ogni passo di traversata, indipendentemente dalla dimensione complessiva del database. La versione Enterprise è stata scelta per la sua capacità di operare in configurazione cluster con topologia primary-secondary, che consente di valutare l'impatto della scalabilità orizzontale sulle prestazioni di lettura.

Il linguaggio Cypher, nativamente orientato all'attraversamento di percorsi su grafi, permette di esprimere in modo intuitivo le query di estrazione dei sottografi.

3.3.2 Libreria Graph Data Science

La libreria GDS estende Neo4j con implementazioni parallele di algoritmi analitici su grafi, esposti come procedure Cypher. Il suo meccanismo centrale è la **graph projection**: prima dell'esecuzione di un algoritmo, il sottografo di interesse viene proiettato in una struttura in memoria separata, denominata *graph catalog*, ottimizzata per il calcolo analitico. Questa proiezione crea una snapshot isolata del grafo, consentendo l'esecuzione di algoritmi intensivi in parallelo senza lock sul database transazionale. Il graph catalog supporta operazioni di filtro e trasformazione delle proprietà dei nodi e degli archi direttamente in fase di caricamento, riducendo il pre-processing necessario nella pipeline di machine learning. Questo separa nettamente il layer transazionale da quello analitico, permettendo l'esecuzione di algoritmi intensivi senza interferire con le operazioni correnti sul database e garantendo al contempo che le proiezioni possano essere riutilizzate per molteplici algoritmi senza ripetere il caricamento.

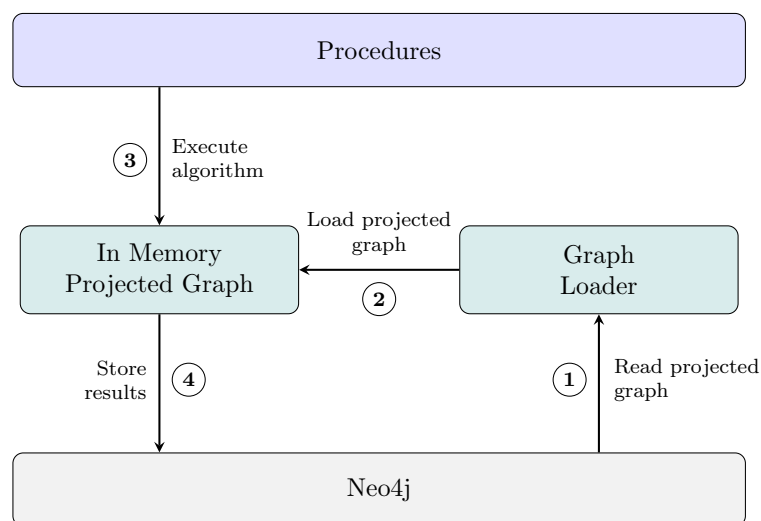


Figura 3.1: Ciclo di vita dell'esecuzione degli algoritmi tramite la libreria Neo4j GDS: il grafo viene proiettato in memoria, l'algoritmo viene eseguito sulla proiezione e i risultati vengono scritti nel database.

3.3.3 PyTorch Geometric

PyTorch Geometric [8] è il framework di riferimento per l'implementazione dei tre modelli di Graph Machine Learning. Sviluppato come estensione di PyTorch, fornisce primitive efficienti per la rappresentazione di grafi eterogenei attraverso il formato **HeteroData**, layer convoluzionali dedicati (come **SAGEConv**) e utility per la suddivisione del dataset in training, validazione e test. Il supporto nativo al calcolo tensoriale su GPU e la possibilità di definire meccanismi di message passing personalizzati lo rendono particolarmente adatto per sperimentare con architetture diverse su grafi di larga scala.

3.3.4 Containerizzazione e automazione infrastrutturale

L'ambiente Neo4j su ciascun nodo del cluster è containerizzato tramite **Docker**, garantendo isolamento delle dipendenze e riproducibilità dell'esecuzione indipendentemente dal sistema operativo sottostante. **Docker Compose** coordina la definizione e l'avvio dei servizi tramite un unico file YAML, semplificando il deployment e la gestione delle versioni del database.

La gestione orchestrata del cluster è affidata ad **Ansible**, che tramite playbook dichiarativi eseguiti via SSH consente di distribuire configurazioni, avviare e fermare i container, sincronizzare file tra i nodi ed eseguire il benchmark nelle diverse configurazioni con un singolo comando. L'assenza di agenti e l'uso di un linguaggio YAML leggibile garantiscono riproducibilità e coerenza tra gli ambienti.

Le quattro macchine virtuali che costituiscono il cluster sono ospitate su **Microsoft Azure**, ciascuna con 8 core virtuali e 32 GB di RAM, all'interno di una stessa rete virtuale per garantire bassa latenza nelle comunicazioni intra-cluster. La scelta di Azure consente di sfruttare la scalabilità elastica e la possibilità di replicare l'infrastruttura in diverse regioni per futuri esperimenti.

I dettagli implementativi della configurazione, inclusi i comandi di deployment, le impostazioni di memoria e la gestione degli indirizzi IP su WSL2, sono riportati nel capitolo di Implementazione (Sezione 4).

3.4 Il dataset: PharMeBiNet V3

PharMeBiNet (*Pharmacological Medical Biochemical Network*) è un knowledge graph biomedico open-source [2] che integra informazioni provenienti da oltre quaranta sorgenti farmacologiche, genomiche e cliniche, tra cui DrugBank, ClinVar, CTD, DisGeNET e UniProt. Il progetto nasce come estensione di Hetionet e ne estende la copertura semantica con ulteriori tipi di entità e relazioni.

La versione **V3** [9] è quella adottata in questo lavoro. Come illustrato in Figura 3.2, il grafo modella un'ampia varietà di entità biologiche come: farmaci, geni, proteine, malattie, varianti genomiche e reazioni avverse e le relazioni che le interconnettono, quali interazioni farmaco-farmaco, associazioni gene-malattia e percorsi metabolici.

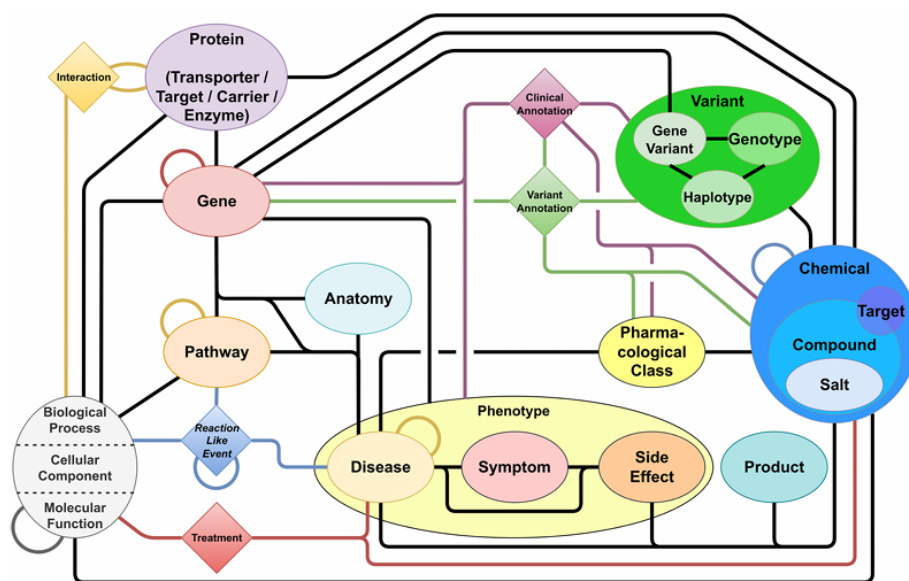


Figura 3.2: Schema del KG PharMeBiNet [1].

Capitolo 4

Implementazione

4.1 Configurazione dell'Ambiente e del Cluster Neo4j

L'infrastruttura sperimentale è distribuita su quattro macchine virtuali Microsoft Azure, ciascuna equipaggiata con Windows 10 all'interno del quale è stato installato il *Windows Subsystem Linux V2* con Ubuntu 22.04 LTS. Le macchine hanno 8 vcore e 32 GB di RAM. La scelta di WSL2 come layer di virtualizzazione ha permesso di eseguire container Docker nativi su Linux mantenendo performance comparabili a un ambiente Linux nativo e con la possibilità di configurare il sistema in modo più aperto. Neo4j Enterprise 2026.04.0 gira all'interno di container Docker su WSL2, orchestrati tramite un file `docker-compose.yml` condiviso tra tutti i nodi del cluster, che definisce i servizi, le variabili d'ambiente e i volumi persistenti per ciascuna istanza.

La topologia adottata prevede un nodo primary e tre nodi secondary. Il primary gestisce tutte le operazioni di scrittura e coordina il consenso distribuito tramite il protocollo Raft, mentre il router integrato di Neo4j gestisce il bilanciamento del carico delle letture tra le secondary. I secondary sono dedicati esclusivamente alla lettura distribuita e al calcolo analitico, senza partecipare alle operazioni di scrittura. La scelta di quattro nodi totali non è motivata dalla fault tolerance ma dall'obiettivo di misurare l'impatto della scalabilità orizzontale sui tempi di estrazione dei dati e di valutare il comportamento del routing layer in presenza di molteplici repliche.

Ogni container Neo4j è stato configurato con limiti di memoria espliciti, bilanciando le esigenze del garbage collector della Java Virtual Machine e la velocità di accesso ai dati tramite page cache:

```
1 server.memory.heap.initial_size=8G
2 server.memory.heap.max_size=8G
3 server.memory.pagecache.size=6G
```

Listing 4.1: Configurazione memoria Neo4j (neo4j.conf)

I limiti a livello Docker sono stati impostati a 20 GB per container, con 6 GB riservati al sistema operativo WSL2. L'heap da 8 GB è dedicato al garbage collector della JVM, mentre i 6 GB di page cache accelerano le letture mantenendo in memoria le porzioni del grafo più frequentemente accedute. Questa suddivisione è stata calibrata per bilanciare il throughput delle query, garantendo al contempo che il garbage collector non introduca pause significative durante le operazioni di lettura intensiva sul sottografo.

L'adozione di WSL2 ha introdotto una complicazione tecnica rilevante: l'indirizzo IP interno assegnato da WSL2 cambia ad ogni riavvio del sistema operativo Windows, rendendo instabili le regole di port-forwarding configurate staticamente. Il problema è stato risolto tramite uno script PowerShell automatizzato, eseguito all'avvio di ogni macchina con privilegi di amministratore, che interroga WSL2 per ottenere l'IP corrente e aggiorna dinamicamente le regole di port-forwarding tramite `netsh` sulle porte necessarie alla comunicazione intra-cluster (6000 per le transazioni, 7000 per il protocollo Raft, 7687 per il protocollo Bolt, 22 per l'SSH), garantendo che il cluster mantenga la connettività intra-cluster anche dopo riavvii non pianificati.

L'intera gestione del cluster è automatizzata tramite Ansible, eseguito dalla primary via SSH verso le secondary. I playbook principali sono `deploy.yml` per distribuire le configurazioni e avviare i container, `stop.yml` per fermarli in modo ordinato, `reset.yml` per un reset completo e `setup.yml` per la creazione della struttura di cartelle. Le variabili specifiche di ciascun nodo, come nome, ruolo e indirizzo IP, sono isolate in un file `.env` generato da Ansible tramite un template Jinja2, garantendo che la configurazione sia riproducibile e centralizzata senza interventi manuali su ogni macchina.

L'uso di Ansible consente inoltre di eseguire il benchmark nelle diverse configurazioni (1, 2 o 4 nodi) in modo sistematico, variando il parametro `nodes` al momento dell'esecuzione del playbook. Questo approccio garantisce che ogni configurazione venga testata nelle stesse condizioni operative, eliminando possibili fonti di variabilità legate a interventi manuali sul cluster. La riproducibilità delle misurazioni è ulteriormente assicurata dall'esecuzione di un set di query di riscaldamento prima di ogni benchmark, che porta il database a regime termico.

La gestione delle dipendenze Python è affidata a `uv`, un package manager ad alte prestazioni che parallelizza il download e la risoluzione delle dipendenze, garantendo riproducibilità tramite un file `pyproject.toml`. Rispetto ai tradizionali gestori di pacchetti, `uv` riduce significativamente i tempi di installazione, un aspetto non trascurabile quando si lavora con ambienti di sviluppo containerizzati e si devono replicare frequentemente gli esperimenti. Le librerie necessarie alla pipeline sono installate con un singolo comando:

```
1 uv add torch torch_geometric neo4j \  
2     pandas numpy scikit-learn
```

Listing 4.2: Installazione delle librerie della pipeline sperimentale

Gli script Python vengono eseguiti tramite `uv run`, che attiva automaticamente l'ambiente virtuale isolato prima dell'esecuzione, eliminando conflitti tra dipendenze di progetti diversi. Questo approccio garantisce che l'ambiente di esecuzione sia sempre coerente con le specifiche dichiarate, semplificando la collaborazione e la condivisione del codice.

4.2 Acquisizione e Ingestion del Dataset

Il dataset è distribuito pubblicamente tramite Zenodo [9] in formato dump nativo Neo4j, ovvero un archivio binario che rappresenta lo stato completo del database a grafo e può essere caricato direttamente tramite il tool `neo4j-admin` senza richiedere alcuna fase di conversione o pre-processing.

Questo formato garantisce che il caricamento preservi integralmente la struttura del grafo, inclusi gli indici e i vincoli di integrità definiti nel database originale. Il file dump è stato scaricato sulla macchina primary e copiato nella cartella **data/raw/** del progetto.

Il caricamento è avvenuto tramite un container Docker effimero della stessa immagine Neo4j, che accede direttamente al layer di storage bypassando il motore transazionale, la modalità più efficiente per l'importazione massiva di dati. Questa strategia evita l'overhead delle transazioni e riduce il tempo complessivo di caricamento, consentendo di importare il grafo da 50 GB in tempi compatibili con lo sviluppo sperimentale. Al termine del caricamento, il container viene automaticamente rimosso, lasciando il database pronto per la successiva fase di configurazione del cluster.

```
1 docker run --rm -it \  
2   -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \  
3   -v $(pwd)/cluster/data:/data \  
4   -v $(pwd)/data/raw:/backups \  
5   neo4j:2026.04.0-enterprise \  
6   neo4j-admin database load pharnebinet \  
7     --from-path=/backups/ \  
8     --overwrite-destination=true
```

Listing 4.3: Caricamento del dump PharMeBiNet V3

Al termine del caricamento sulla primary, il database è stato distribuito automaticamente sulle secondary tramite il meccanismo di seeding nativo di Neo4j Cluster. La distribuzione avviene tramite una query Cypher che istruisce il cluster a replicare i dati esistenti a partire dal nodo sorgente specificato:

```
1 CREATE DATABASE pharnebinet  
2   TOPOLOGY 1 PRIMARY 3 SECONDARY  
3   OPTIONS {  
4     existingData: 'use',  
5     existingDataSeedServer: '<SERVER_ID_PRIMARY>'};
```

Listing 4.4: Creazione del database distribuito con seeding

Il processo di replica di circa 50 GB ha richiesto diversi minuti, al termine dei quali tutti e quattro i nodi risultavano **online** per il database **pharmebinet**, pronti a servire sia operazioni di scrittura sul primary sia di lettura sui secondary. Una volta completata la replica, è stata verificata la consistenza del grafo attraverso query di conteggio sui nodi e sulle relazioni, confermando che il dataset era stato distribuito correttamente senza perdita di informazioni.

4.3 Data Preprocessing ed Estrazione del Sottografo

Prima di procedere con la sperimentazione, il grafo è stato sottoposto a un'analisi esplorativa sistematica tramite query Cypher eseguite su Neo4j Browser, con l'obiettivo di caratterizzarne la struttura topologica e di identificare il sottografo tematico più adatto al task di link prediction.

```
1 CALL apoc.meta.stats()
2 YIELD labels, relTypesCount, nodeCount, relCount
3 RETURN labels, relTypesCount, nodeCount, relCount
```

Listing 4.5: Query EDA su PharMeBiNet

I risultati confermano la forte eterogeneità del dataset. I nodi dominanti per cardinalità sono le varianti genomiche (**Variant**, **GeneVariant**, **SNV**, circa 3,8 milioni ciascuno), seguiti da modificazioni post-traduzionali (**PTS**, 782 mila), composti chimici (**Chemical**, 594 mila) e interazioni (**Interaction**, 1,1 milioni). I nodi **Disease** ammontano a 23.580 e i **Gene** a circa 193 mila. L'analisi delle relazioni più dense ha evidenziato che la coppia **Chemical**→**Chemical** RESEMBLES domina il grafo con 31 milioni di archi, circa il 58% del totale. Per il task di link prediction, la coppia **Chemical**→**Disease** è stata selezionata per la sua rilevanza biologica nel contesto del drug repurposing e per la disponibilità di dati sufficienti: 123.361 relazioni tra 594.343 nodi Chemical e 23.580 nodi Disease. Questa scelta consente di valutare i modelli su un task concreto e clinicamente rilevante, mantenendo al contempo un volume di dati gestibile per l'addestramento dei modelli di GML.

Una seconda query ha analizzato il grado medio e massimo dei principali tipi di nodo, al fine di valutare l’impatto della densità locale sulle scelte algoritmiche:

```

1 MATCH (n)
2 WHERE n:Disease OR n:Chemical OR n:Gene OR n:Protein
3 WITH labels(n)[0] AS tipo, count(*) AS totale,
4      avg(size([(n)--() | 1])) AS grado_medio,
5      max(size([(n)--() | 1])) AS grado_max
6 RETURN tipo, totale, round(grado_medio) AS grado_medio, grado_max
7 ORDER BY grado_medio DESC

```

Listing 4.6: Query sul grado dei nodi principali

Tipo	Totale nodi	Grado medio	Grado massimo
Protein	20.420	232	46.069
Chemical	594.306	112	25.034
Disease	23.580	47	30.419
Gene	193.613	46	36.892

Tabella 4.1: Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3.

I gradi massimi sono molto elevati: un singolo nodo Protein raggiunge oltre 46.000 connessioni, mentre Chemical e Disease superano rispettivamente 25.000 e 30.000 archi incidenti. Questo ha implicazioni dirette sulla scelta degli algoritmi: GraphSAGE, che campiona un numero fisso di vicini per contenere il costo computazionale, rischia di introdurre rumore informativo su nodi ad alto grado, poiché il campionamento casuale può escludere vicini semanticamente rilevanti o sovrarappresentare porzioni meno significative del vicinato. HashGNN, che mantiene proiezioni separate per tipo di nodo, è teoricamente più adatto a gestire questa eterogeneità strutturale, anche se il suo vantaggio dipende dalla capacità dei layer lineari di compensare la binarizzazione introdotta dalle proiezioni hash. TransE, non richiedendo aggregazione di vicinato, risulta immune a questo problema ma rinuncia completamente all’informazione strutturale locale, affidandosi esclusivamente alla geometria delle relazioni nello spazio latente.

L'estrazione del sottografo verso PyTorch Geometric [8] è implementata nel codice tramite il file `src/extractor.py`, utilizzando il driver ufficiale Neo4j per Python, che gestisce la connessione al cluster e l'esecuzione delle query Cypher. Il driver utilizza il protocollo Bolt su porta 7687, garantendo comunicazioni efficienti e tipizzate tra il database e l'ambiente di machine learning. La pipeline esegue tre query Cypher sequenziali che estraggono rispettivamente i nodi Chemical, i nodi Disease e le relazioni dirette tra loro:

```
1 MATCH (c:Chemical) RETURN DISTINCT c.identifier AS id
2
3 MATCH (d:Disease) RETURN DISTINCT d.identifier AS id
4
5 MATCH (c:Chemical)-[r]->(d:Disease)
6 RETURN c.identifier AS src, d.identifier AS dst
```

Listing 4.7: Query di estrazione del sottografo Chemical→Disease

Per ciascun tipo di nodo viene costruito un dizionario di mapping che traduce gli identificatori testuali in indici interi contigui a partire da zero, operazione necessaria poiché PyTorch Geometric opera esclusivamente su tensori numerici. Il risultato viene incapsulato in un oggetto `HeteroData`:

```
1 data['Chemical'].num_nodes = len(chem_mapping)
2 data['Disease'].num_nodes = len(dis_mapping)
3 data['Chemical', 'relates_to', 'Disease'].edge_index = \
4     torch.tensor([src_idx, dst_idx], dtype=torch.long)
```

Listing 4.8: Costruzione dell'oggetto `HeteroData` per PyTorch Geometric

L'`edge_index` è un tensore di forma `[2, num_edges]` in formato COO (*Coordinate Format*): la prima riga contiene gli indici dei nodi sorgente e la seconda quelli dei nodi destinatari. Questo formato sparso è lo standard in PyTorch Geometric per rappresentare topologie di rete, garantendo un'occupazione di memoria lineare rispetto al numero di archi.

4.4 Implementazione dei Modelli di Embedding

Poiché i nodi Chemical e Disease non possiedono feature numeriche nel grafo ma sono identificati solo da stringhe testuali, tutti e tre i modelli adottano **embedding learnable** come feature iniziali: tabelle di vettori inizializzati casualmente e ottimizzati durante il training tramite backpropagation, una per ogni tipo di nodo. Man mano che il modello minimizza la funzione di perdita, questi vettori si spostano nello spazio latente fino a codificare l'identità relazionale e le proprietà topologiche di ciascuna entità. La dimensione degli embedding è un iperparametro che bilancia la capacità rappresentativa del modello e il costo computazionale: per tutti i modelli è stata scelta una dimensione pari a 64 per i layer nascosti e 32 per gli output finali.

GraphSAGE è implementato in `src/models/graphsage.py` tramite i layer **SAGEConv** di PyTorch Geometric. L'architettura prevede due layer convoluzionali impilati, che consentono a ciascun nodo di aggregare informazioni dal vicinato fino a due salti di distanza. Gli embedding learnable dei due tipi di nodo vengono fusi in un unico tensore omogeneo tramite concatenazione, prima di essere passati ai layer convoluzionali, con gli indici Disease traslati di `num_chemical` posizioni per evitare collisioni nello spazio globale. Questa fusione iniziale è una scelta progettuale fondamentale: costringe le due tipologie di entità a condividere lo stesso spazio geometrico fin dalle fasi iniziali del training, riducendo la complessità del compito di apprendimento per il decoder MLP successivo. Il codice seguente illustra il meccanismo di offset applicato agli archi del grafo:

```
1 x = torch.cat([self.chem_emb.weight, self.dis_emb.weight], dim=0)
2 offset = torch.zeros_like(edge_index)
3 offset[1] = num_chemical
4 edge_index_global = edge_index + offset
```

Listing 4.9: Fusione degli spazi di embedding con offset in GraphSAGE

Due layer **SAGEConv** propagano l'informazione strutturale del vicinato, il primo a 1 salto e il secondo a 2 salti, con attivazione ReLU intermedia. L'encoder produce una matrice $\mathbf{Z} \in \mathbb{R}^{(N_A+N_B) \times d}$ con gli embedding aggiornati di tutti i nodi.

TransE è implementato in `src/models/transe.py`. A differenza di GraphSAGE, non utilizza alcuna forma di aggregazione di vicinato: mantiene tre tabelle di embedding separate per i nodi Chemical, i nodi Disease e i tipi di relazione, inizializzate con distribuzione uniforme in $[-1, 1]$. Durante il forward pass, tutti gli embedding vengono normalizzati L2 prima del calcolo del punteggio, per evitare che il modello raggiunga una loss bassa espandendo artificialmente le norme dei vettori:

```
1 def _score(self, src, rel, dst):
2     h = F.normalize(self.chem_emb(src), dim=-1)
3     r = F.normalize(self.rel_emb(rel), dim=-1)
4     t = F.normalize(self.dis_emb(dst), dim=-1)
5     return torch.norm(h + r - t, p=2, dim=-1)
```

Listing 4.10: Funzione di scoring di TransE

Il training usa una margin ranking loss con margine $\gamma = 1.0$, che penalizza i casi in cui il punteggio di una tripla negativa non supera quello di una tripla positiva di almeno γ , inducendo una dinamica di attrazione e repulsione nello spazio latente.

HashGNN è implementato in `src/models/hashgnn.py`. La differenza architetturale fondamentale rispetto a GraphSAGE è che gli embedding dei due tipi di nodo non vengono mai fusi in un unico spazio omogeneo durante l'encoding. Le proiezioni hash sono matrici casuali fisse registrate come buffer PyTorch non aggiornabili, che non partecipano all'ottimizzazione ma vengono correttamente salvate con il modello:

```
1 def hash_features(self, x):
2     return torch.cat([
3         torch.sign(x @ self.hash_weights[i].T)
4         for i in range(self.num_hashes)
5     ], dim=-1)
```

Listing 4.11: Funzione di hashing LSH in HashGNN

La binarizzazione tramite `torch.sign` produce K viste binarie dello stesso nodo, ciascuna che cattura aspetti complementari della struttura locale. Layer lineari separati per tipo di nodo trasformano queste rappresentazioni nell'embedding finale.

4.5 Implementazione del Classificatore Binario (MLP)

I tre modelli condividono la stessa architettura di decoder: un Multi-Layer Perceptron che riceve in input gli embedding concatenati di una coppia di nodi e restituisce uno score di probabilità tramite funzione sigmoidale. L'architettura del MLP è composta da due layer lineari intervallati da una attivazione ReLU, con un layer nascosto di dimensione pari a 64 neuroni. È stato inoltre aggiunto un tasso di dropout del 20% tra i due layer lineari per ridurre il rischio di overfitting, considerato il numero limitato di archi disponibili per il training. La scelta di un MLP al posto di un semplice prodotto scalare è motivata da due ragioni. Il prodotto scalare è un'operazione simmetrica, trattando la relazione Chemical→Disease in modo identico alla direzione inversa, mentre un MLP con vettori concatenati in ordine preciso preserva la direzionalità causale della relazione, un aspetto cruciale in ambito biomedico dove l'effetto di un farmaco su una malattia non è reversibile. Inoltre, Chemical e Disease sono entità biologicamente distinte: un MLP agisce da traduttore tra i loro spazi di rappresentazione, apprendendo la regola di compatibilità senza richiedere che i due tipi di nodo abbiano embedding geometricamente simili. Questa flessibilità è particolarmente vantaggiosa quando gli spazi di partenza sono intrinsecamente diversi, come nel caso di HashGNN, e consente al modello di adattarsi a distribuzioni dei dati non allineate.

Per GraphSAGE e TransE il decoder riceve embedding provenienti da uno spazio condiviso, semplificando il compito di apprendimento poiché le due entità condividono già le stesse coordinate geometriche. Per HashGNN i due embedding provengono da spazi separati e si incontrano per la prima volta nel decoder, che deve quindi apprendere anche la mappatura tra i due spazi eterogenei, un compito aggiuntivo che può rallentare la convergenza ma che teoricamente consente di preservare una maggiore specificità semantica per ciascun tipo di entità. La funzione sigmoidale finale trasforma lo score continuo in una probabilità $p \in [0, 1]$, consentendo di interpretare l'output del modello come una misura di confidenza sull'esistenza del link, con soglia di classificazione tipicamente fissata a 0.5 per bilanciare precisione e richiamo.

4.6 Definizione della Pipeline Sperimentale

La pipeline sperimentale è automatizzata dallo script `run_pipeline.sh`, che esegue in sequenza il benchmark infrastrutturale nelle tre configurazioni e il training dei tre modelli per cinque run indipendenti, salvando i risultati in `results/benchmark.csv` e `results/model_results_final.csv`.

Prima del training, il dataset viene suddiviso tramite `RandomLinkSplit` di PyTorch Geometric:

```
1 transform = T.RandomLinkSplit(  
2     num_val=0.0,  
3     num_test=0.1,  
4     is_undirected=False,  
5     edge_types=[('Chemical', 'relates_to', 'Disease')],  
6     add_negative_train_samples=False,  
7 )  
8 train_data, _, test_data = transform(data)
```

Listing 4.12: Suddivisione training/test con `RandomLinkSplit`

Il 10% degli archi viene tenuto completamente nascosto durante il training e usato esclusivamente per la valutazione finale. I campioni negativi vengono ricreati casualmente ad ogni epoca, introducendo variabilità e riducendo il rischio di overfitting sui negativi. Per tutti e tre i modelli la procedura di training è identica: calcolo delle predizioni su link positivi, generazione di campioni negativi casuali in numero uguale, calcolo della loss, backpropagation e aggiornamento dei pesi tramite Adam optimizer. Dopo 200 epoche, valutazione sul test set con archi mai visti durante il training.

Per garantire la riproducibilità statistica, ogni esperimento è stato ripetuto cinque volte con seed diverso ($42 + i$ con $i \in \{0, 1, 2, 3, 4\}$), variando sia l'inizializzazione dei pesi sia la generazione dei campioni negativi. Questo permette di stimare la varianza dei risultati e distinguere differenze sistematiche tra i modelli da fluttuazioni casuali legate all'inizializzazione.

Iperparametro	Valore
Epoche	200
Learning rate	10^{-3}
Optimizer	Adam
Hidden channels	64
Output channels	32
Test split	10%
Run indipendenti	5
Seed iniziale	42

Tabella 4.2: Iperparametri condivisi tra i tre modelli.

Il benchmark infrastrutturale misura il tempo totale di estrazione nelle tre configurazioni del cluster: connessione diretta alla primary tramite `bolt://10.0.0.5:7687` per il caso a 1 nodo, e connessione tramite routing distribuito `neo4j://10.0.0.5:7687` per i casi a 2 e 4 nodi. Per la configurazione a 2 nodi, le secondary 2 e 3 vengono spente tramite Ansible prima della misurazione e riaccese al termine, attendendo 30 secondi per permettere al cluster di riorganizzare la routing table.

Prima di eseguire la pipeline è necessario caricare le variabili d'ambiente dal file `.env`:

```

1 export $(grep -v '^#' .env | xargs)
2 ./run_pipeline.sh

```

Listing 4.13: Esecuzione della pipeline completa

I script Python leggono la password del database tramite `os.getenv("DB_PASSWORD")` senza default hardcoded nel codice, per evitare che credenziali sensibili vengano accidentalmente incluse nel sistema di versionamento.

Capitolo 5

Risultati e Discussione

5.1 Panoramica dei risultati

Questo capitolo presenta e discute i risultati ottenuti dall'esecuzione della pipeline sperimentale descritta nel capitolo precedente. L'analisi si articola su due livelli distinti e complementari.

Il primo livello riguarda la valutazione infrastrutturale: vengono misurati i tempi di estrazione del sottografo da Neo4j verso PyTorch Geometric al variare del numero di nodi attivi nel cluster, con l'obiettivo di quantificare l'impatto concreto della scalabilità orizzontale su un carico di lavoro reale di data engineering.

Il secondo livello riguarda la valutazione predittiva: vengono confrontate le prestazioni dei tre modelli di GML in termini di accuratezza sulla link prediction e di efficienza computazionale durante il training, con l'obiettivo di determinare quale approccio, induttivo o trasduttivo, si adatta meglio alle caratteristiche strutturali del sottografo analizzato.

I risultati vengono infine discussi in relazione alle ipotesi formulate prima degli esperimenti, evidenziando dove le previsioni teoriche si sono confermate e dove invece i dati hanno mostrato comportamenti controintuitivi, fornendo in entrambi i casi una spiegazione architetturale e sistemistica delle cause osservate.

5.2 Benchmark infrastrutturale

5.2.1 Tempi di estrazione al variare dei nodi attivi

Il benchmark infrastrutturale misura il tempo complessivo necessario per estrarre l'intero sottografo Chemical→Disease da Neo4j e costruire l'oggetto `HeteroData` per PyTorch Geometric [8], al variare del numero di nodi attivi del cluster. Questo tempo di estrazione rappresenta un collo di bottiglia critico in pipeline di Graph Machine Learning su larga scala, poiché determina la frequenza con cui è possibile aggiornare i modelli al variare dei dati sottostanti. Ogni misurazione include le tre query Cypher sequenziali rispettivamente, nodi Chemical, nodi Disease e relazioni, oltre alla costruzione del tensore `edge_index`, che trasforma le relazioni estratte in un formato compatibile con il calcolo tensoriale su GPU. Per garantire la ripetibilità delle misurazioni, il benchmark viene eseguito dopo un periodo di riscaldamento del database che porta la page cache a regime, eliminando l'effetto di latenze transitorie dovute al primo accesso ai dati. I risultati sono ottenuti dall'esecuzione della pipeline completa con il cluster a regime, dopo il caricamento e la distribuzione di PharMeBiNet V3, assicurando che il sistema si trovi in condizioni operative stabili e rappresentative di uno scenario d'uso reale.

Configurazione	Nodi attivi	Protocollo	Tempo medio (s)
1 nodo	Primary only	<code>bolt://</code>	33.8
2 nodi	Primary + Secondary 1	<code>neo4j://</code>	32.1
4 nodi	Cluster completo	<code>neo4j://</code>	33.5

Tabella 5.1: Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.

Configurazione	Tempo (s)	Variazione	Nodi attivi
1 nodo	33.8	baseline	1
2 nodi	32.1	-5.0%	2
4 nodi	33.5	-0.9%	4

Tabella 5.2: Variazione percentuale dei tempi di estrazione rispetto al baseline a 1 nodo.

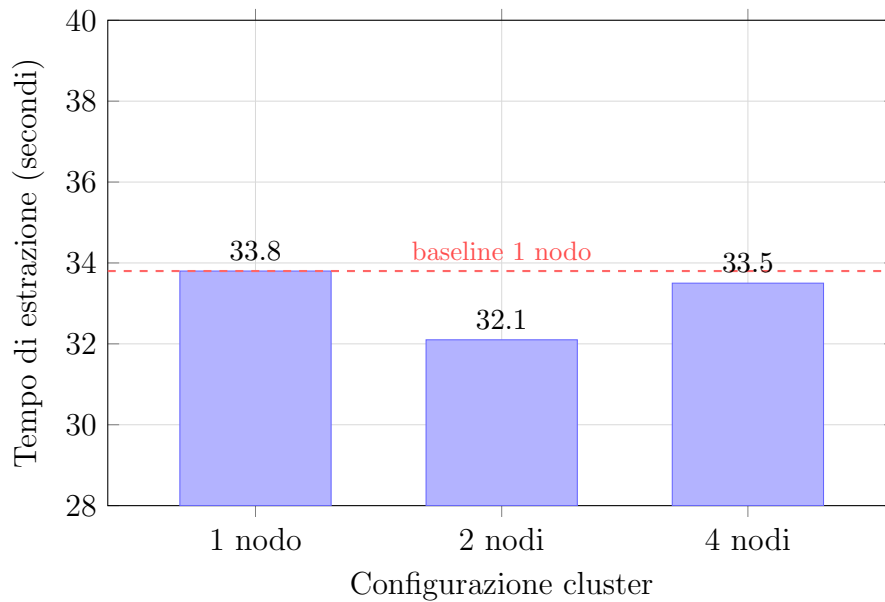


Figura 5.1: Tempi di estrazione nelle tre configurazioni del cluster. La linea tratteggiata evidenzia il baseline a 1 nodo.

5.2.2 Analisi dei risultati infrastrutturali

I tempi risultano sostanzialmente costanti tra le tre configurazioni, con una variazione massima di circa 1.7 secondi. Questo risultato, che potrebbe sembrare deludente, è in realtà coerente con la natura del carico di lavoro e con i principi fondamentali dei sistemi distribuiti.

Il bottleneck non è la disponibilità di nodi computazionali, ma la **capacità di I/O della singola query sequenziale** su un dataset da circa 50 GB. Le tre query Cypher vengono eseguite una dopo l'altra: non esiste parallelismo a livello applicativo che possa sfruttare la presenza di più nodi. Il router Neo4j distribuisce le letture sulle secondary tramite la routing table, ma con una sola connessione attiva alla volta il vantaggio è marginale e viene parzialmente annullato dall'overhead di coordinazione del cluster.

Un dato interessante emerge dal confronto tra le configurazioni a 2 e 4 nodi: la configurazione a 2 nodi risulta leggermente più veloce della configurazione a 4 nodi, come evidenziato nella tabella 5.2.

Questo comportamento suggerisce l'esistenza di un *sweet spot* architetturale: con 2 nodi il bilanciamento tra parallelismo di lettura e overhead di routing è ottimale, mentre l'aggiunta di ulteriori secondary introduce latenze di coordinazione che superano il vantaggio.

Va sottolineato che questo risultato non inficia il valore del cluster. Neo4j Enterprise in configurazione distribuita offre vantaggi significativi che questo benchmark non misura direttamente:

- **Fault tolerance:** con la topologia 1 primary e 3 secondary, il servizio rimane operativo anche in caso di guasto di un nodo, senza interruzione delle letture.
- **Disponibilità:** il routing automatico garantisce la continuità delle operazioni anche durante manutenzioni programmate.
- **Scalabilità su carichi paralleli:** con più client che eseguono query simultanee, il cluster distribuisce il carico sulle secondary producendo speedup significativi, uno scenario non testato in questa tesi ma teoricamente più favorevole.

5.3 Risultati della Link Prediction

5.3.1 Metriche per modello

Il training è stato eseguito per **200 epoche** su **5 run indipendenti**. Le 200 epoche sono state selezionate per garantire la convergenza di tutti i modelli, verificata monitorando l'andamento della loss. Ciascun modello è stato valutato sul 10% degli archi tenuti nascosti durante il training tramite **RandomLinkSplit**, un bilanciamento tra capacità di generalizzazione e dati disponibili sufficienti per una valutazione statisticamente significativa. L'uso di run indipendenti con semi diversi consente di verificare che le differenze prestazionali tra i modelli non siano artefatti dell'inizializzazione casuale dei pesi o della generazione dei campioni negativi.

Modello	AUC-ROC medio	Std	Tempo medio (s)	Std (s)
GraphSAGE	0.9934	0.0003	448.7	48.2
HashGNN	0.9923	0.0002	624.2	62.1
TransE	0.9839	0.0006	145.2	11.8

Tabella 5.3: Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.

Run	GraphSAGE		TransE		HashGNN	
	AUC-ROC	Tempo (s)	AUC-ROC	Tempo (s)	AUC-ROC	Tempo (s)
1	0.9929	471.8	0.9839	150.6	0.9924	629.7
2	0.9935	435.2	0.9841	144.3	0.9926	612.5
3	0.9938	436.5	0.9834	146.5	0.9923	623.1
4	0.9933	454.3	0.9838	145.8	0.9920	625.8
5	0.9934	445.5	0.9841	143.0	0.9924	630.1

Tabella 5.4: Risultati dettagliati per singola run, raggruppati per modello.

5.3.2 Confronto grafico

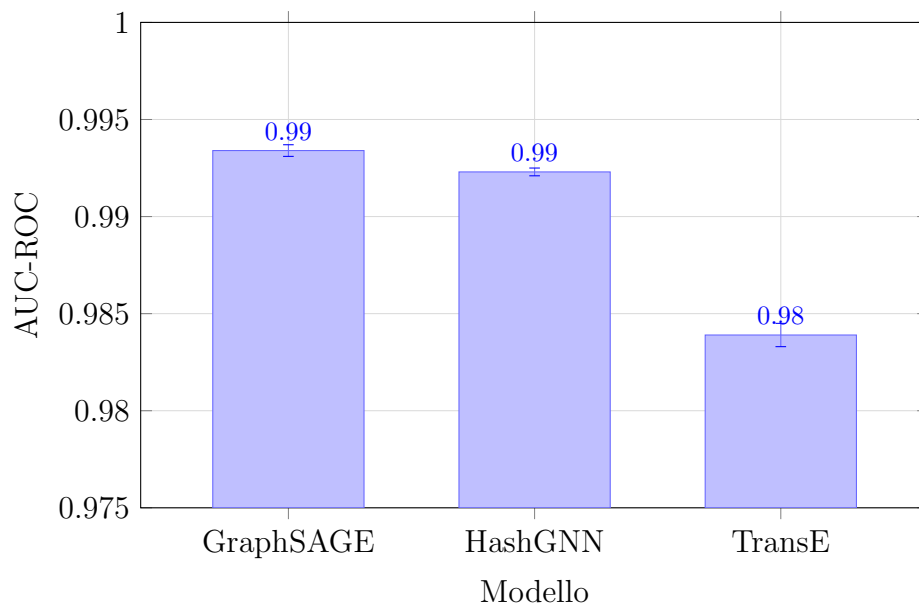


Figura 5.2: AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti. L'asse Y parte da 0.975 per rendere visibili le differenze tra i modelli.

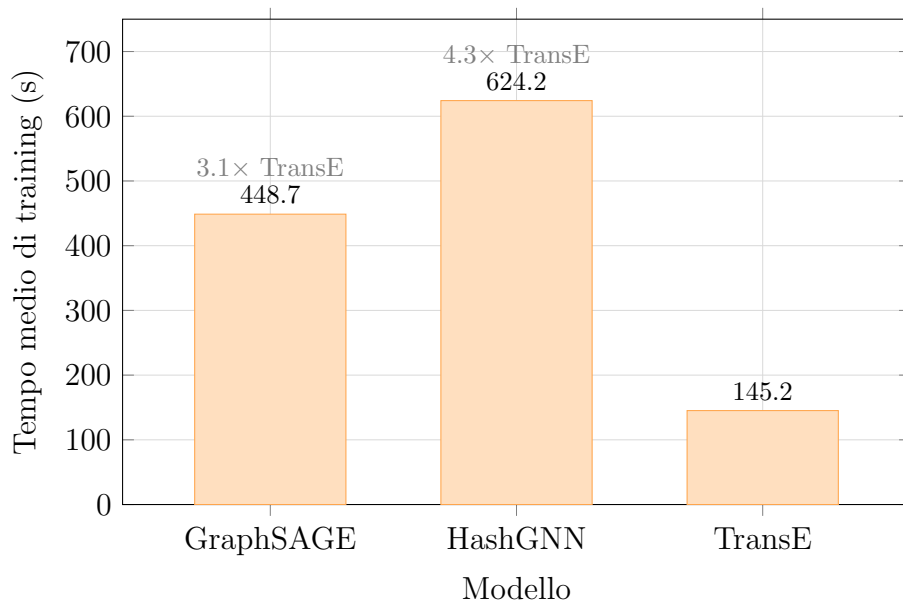


Figura 5.3: Tempo medio di training per i tre modelli su 5 run indipendenti

5.4 Discussione

5.4.1 Analisi delle prestazioni di GraphSAGE

GraphSAGE ha registrato l'accuratezza maggiore, con un AUC-ROC medio pari a **0.9934** e una deviazione standard di 0.0003, denotando un'elevata stabilità statistica nelle diverse esecuzioni. Tale evidenza empirica diverge dall'ipotesi preliminare, che postulava una superiorità di HashGNN in virtù della sua gestione nativa dell'eterogeneità.

Le prestazioni ottimali di GraphSAGE possono essere ricondotte a due fattori. Il primo risiede nella **fusione degli spazi di embedding**: concatenando le rappresentazioni dei due tipi di nodo in unico spazio latente con *offset*, il modello induce le entità a condividere le stesse coordinate geometriche sin dalle fasi iniziali dell'addestramento. Questa convergenza spaziale riduce la complessità a carico *decoder* MLP nell'inferire la probabilità di collegamento. Il secondo fattore riguarda la **velocità di convergenza**: le matrici dei pesi nei layer SAGEConv, dense ottimizzabili gradiente, garantiscono una propagazione del segnale efficiente rispetto alle architetture basate su proiezioni fisse.

Nonostante i risultati ottenuti, occorre ribadire un limite teorico di GraphSAGE già

evidenziato in sede di analisi esplorativa: il modello aggrega i dati del vicinato in modo omogeneo, non distinguendo la tipologia semantica delle relazioni. Sebbene all'interno del sottografo isolato (Chemical→Disease) ciò non costituisca un fattore penalizzante, poiché tutte le relazioni condividono il medesimo significato semantico, l'applicazione su un grafo eterogeneo completo comporterebbe verosimilmente un degrado fisiologico delle prestazioni predittive.

5.4.2 Analisi delle prestazioni di HashGNN

HashGNN si posiziona immediatamente al di sotto di GraphSAGE con un AUC-ROC medio di **0.9923** (scarto di 0.0011). Considerando la deviazione standard (0.0002), le distribuzioni dei risultati dei due modelli risultano statisticamente sovrapponibili in gran parte: in un contesto applicativo legato al *drug repurposing*, tale differenza di accuratezza risulta trascurabile ai fini operativi.

Tuttavia, si osserva una marcata disparità in termini di **complessità temporale**: con un valore medio di 624 secondi per 200 epoche, l'addestramento di HashGNN risulta circa 1.4 volte più oneroso rispetto a GraphSAGE e 4.3 volte superiore rispetto a TransE. Questo incremento è imputabile all'uso delle proiezioni *hash* fisse. I layer lineari successivi richiedono un maggior numero di iterazioni per processare rappresentazioni binarizzate e stocastiche. Poiché le matrici di *hashing* non sono soggette ad aggiornamento via *backpropagation*, l'ottimizzazione grava interamente sui layer lineari dedicati, i quali devono compensare l'iniziale trasformazione aleatoria, rallentando di conseguenza la convergenza.

L'aspettativa teorica di una predominanza di HashGNN non ha trovato riscontro empirico nel perimetro sperimentale analizzato. L'assenza di un vantaggio misurabile suggerisce che, per questo specifico dominio relazionale, l'omofilia biologica intrinseca al sottografo è sufficientemente delineata da poter essere appresa in modo ottimale da un modello omogeneo, rendendo superflua una gestione esplicita dell'eterogeneità.

5.4.3 Analisi delle prestazioni di TransE

TransE ha conseguito un AUC-ROC medio pari a **0.9839**, posizionandosi al di sotto di GraphSAGE di circa un punto percentuale, ma con un tempo di addestramento significativamente ridotto, pari a soli **145 secondi**. Ciò qualifica l'algoritmo come la soluzione computazionalmente più efficiente tra quelle in esame, con un rapporto costo-prestazioni che lo rende particolarmente interessante per scenari operativi con risorse limitate, dove la riduzione dei tempi di calcolo può tradursi in minori costi infrastrutturali e maggiore flessibilità operativa. Tale efficienza è il risultato diretto di un'architettura essenziale: il modello non implementa meccanismi di *message passing* né layer convoluzionali per l'aggregazione del vicinato. L'aggiornamento dei pesi si basa esclusivamente su operazioni di *lookup* all'interno delle matrici di embedding e sul calcolo della distanza L2, calcoli caratterizzati da una bassa complessità asintotica e da un'elevata scalabilità anche su topologie di grandi dimensioni. A differenza delle GNN, che richiedono la propagazione del segnale attraverso l'intero vicinato, TransE opera indipendentemente dalla densità locale del grafo, mantenendo costante il costo per epoca di training, un vantaggio non trascurabile quando si lavora con grafi che presentano nodi con grado estremamente elevato.

È fondamentale osservare che, nel contesto di questo studio, TransE non ha potuto sfruttare appieno la sua capacità di codificare esplicitamente relazioni semanticamente distinte (attraverso l'ottimizzazione del vettore di traslazione $\mathbf{r}$), in quanto gli archi all'interno del sottografo sono stati trattati come appartenenti a un'unica classe relazionale. In un'analisi estesa all'intera topologia multimodale di PharMeBiNet, tale modello potrebbe far emergere in modo più netto i propri vantaggi teorici rispetto agli approcci convoluzionali standard, dove la distinzione semantica delle relazioni viene invece appiattita dall'aggregazione omogenea del vicinato, confermando come la scelta dell'algoritmo debba essere guidata dalla complessità semantica del grafo e dalle priorità operative del contesto applicativo.

5.4.4 Analisi del trade-off tra accuratezza ed efficienza

La Figura 5.4 illustra la distribuzione dei modelli nello spazio dimensionale definito da accuratezza ed efficienza computazionale. L'analisi grafica evidenzia una condizione rilevante: **GraphSAGE domina HashGNN in senso di Pareto**. A parità di fascia di accuratezza, GraphSAGE raggiunge la convergenza in un tempo strettamente inferiore; di conseguenza, all'interno dei confini di questo studio, l'adozione di HashGNN non trova una giustificazione operativa oggettiva rispetto alla controparte. Questo risultato suggerisce che la complessità aggiuntiva introdotta dalla gestione separata degli spazi di embedding non si traduce in un vantaggio predittivo misurabile sul sottografo considerato, probabilmente perché la relazione Chemical-Disease presenta un'omofilia strutturale sufficientemente forte da essere catturata efficacemente da un modello omogeneo senza la necessità di spazi di rappresentazione distinti.

TransE, al contrario, si colloca in un distinto quadrante dello spazio delle soluzioni, configurandosi come un'alternativa operativa indipendente. In contesti industriali caratterizzati da volumi di dati su larga scala o da necessità di riaddestramento frequente, l'accettazione di una lieve flessione nell'accuratezza (circa 0.01 in termini di AUC) costituisce un compromesso metodologico razionale a fronte di un abbattimento dei tempi di esecuzione nell'ordine del 67%. Questo trade-off assume particolare rilevanza in scenari di produzione dove i modelli devono essere aggiornati periodicamente su nuovi dati, rendendo il costo computazionale un fattore critico almeno quanto l'accuratezza predittiva, soprattutto quando le risorse di calcolo sono limitate o i vincoli temporali impongono risposte rapide.

5.4.5 Verifica delle ipotesi sperimentali

L'impianto teorico preliminare postulava un ordinamento decrescente delle prestazioni dominato da HashGNN, seguito da TransE e infine da GraphSAGE. L'evidenza empirica ha restituito una gerarchia differente: GraphSAGE ha mostrato i risultati più solidi, seguito a breve distanza da HashGNN e successivamente da TransE.

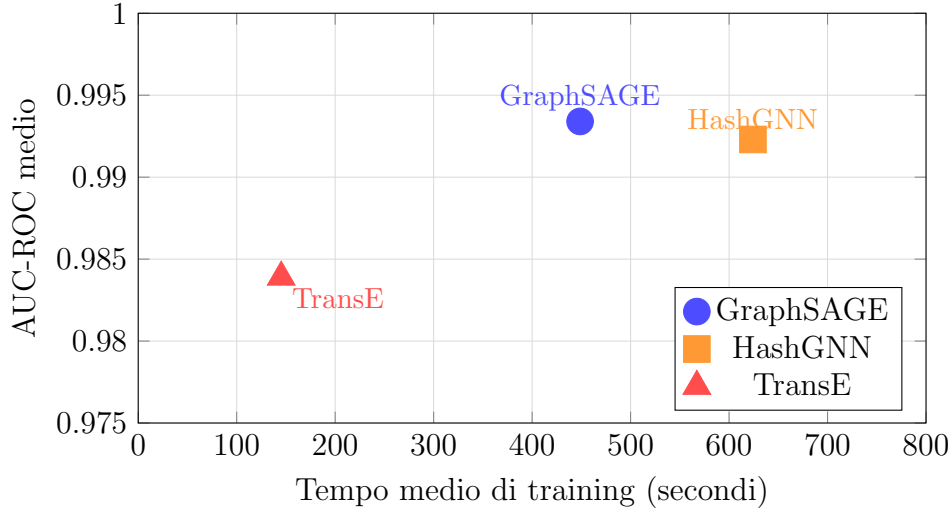


Figura 5.4: Trade-off tra accuratezza ed efficienza computazionale per i tre modelli.

Dal punto di vista infrastrutturale, l'atteso incremento di *throughput* associato all'impiego di un cluster distribuito a 4 nodi rispetto a un'istanza singola non ha trovato conferma. Tale risultato, pienamente coerente con i paradigmi teorici dei sistemi distribuiti per operazioni di I/O, dimostra empiricamente che, in presenza di flussi di estrazione dati sequenziali di tipo *batch*, la scalabilità orizzontale di Neo4j non comporta riduzioni dei tempi di latenza. Il reale beneficio dell'architettura distribuita risiede pertanto nell'alta disponibilità e nella tolleranza ai guasti (*fault tolerance*), non nell'accelerazione intrinseca della singola interrogazione sequenziale.

L'analisi degli scostamenti tra le aspettative teoriche e i riscontri empirici fornisce un elemento di valutazione centrale: attesta che i vantaggi offerti da architetture software o algoritmiche sofisticate necessitano di specifiche condizioni al contorno per potersi manifestare, come la presenza di un grafo nativamente eterogeneo per HashGNN o l'esecuzione di carichi interrogativi fortemente paralleli nel caso del cluster distribuito.

Capitolo 6

Conclusioni

6.1 Sintesi del lavoro

Questo lavoro ha affrontato congiuntamente due problematiche centrali nell’ambito della bioinformatica computazionale e dell’ingegneria dei dati: la valutazione comparativa di architetture di Graph Machine Learning applicate a un *KG* biomedico di grandi dimensioni e la misurazione empirica dell’impatto della scalabilità orizzontale di un database a grafo sui tempi di alimentazione di una pipeline analitica.

Il dataset scelto, PharMeBiNet V3, ha costituito un banco di prova sfidante per via della sua elevata eterogeneità strutturale e semantica. Il dominio applicativo si è concentrato sulla *link prediction* tra composti chimici (*Chemical*) e patologie (*Disease*), un task predittivo di vitale importanza nel contesto del *drug repurposing* finalizzato alla scoperta di relazioni terapeutiche latenti.

Dal punto di vista dell’ingegneria del software, l’infrastruttura sperimentale si è basata su un cluster Neo4j Enterprise distribuito su quattro macchine virtuali cloud in topologia *primary-secondary*. L’intera architettura è stata orchestrata e automatizzata tramite Ansible e container Docker, garantendo la totale riproducibilità degli esperimenti attraverso una pipeline *end-to-end*.

6.2 Discussione dei Risultati

6.2.1 Analisi Comparativa delle Architetture Predittive

Il confronto tra i modelli ha evidenziato risultati controintuitivi rispetto all'ipotesi di partenza. **GraphSAGE** [5] si è dimostrato l'algoritmo più accurato, registrando un AUC-ROC medio di 0.9934. Il successo di un approccio architetturale omogeneo su un grafo eterogeneo si spiega con la forte omofilia biologica presente nel sottografo estratto: la proiezione forzata delle *feature* in un unico spazio tensoriale condiviso ha semplificato l'ottimizzazione del *decoder* MLP, favorendo una convergenza rapida e precisa.

HashGNN [6], pur essendo progettato nativamente per gestire l'eterogeneità topologica, ha ottenuto un AUC-ROC medio di 0.9923. Tale scarto (0.0011) risulta statisticamente trascurabile, ma si accompagna a tempi di addestramento circa 1.4 volte superiori rispetto a GraphSAGE. Ne consegue che il vantaggio teorico offerto dal partizionamento dei messaggi per tipologia di nodo non si traduce in un reale guadagno prestazionale sul problema e sul perimetro dati analizzato.

Infine, **TransE** [4] ha riportato un'accuratezza leggermente inferiore (AUC-ROC 0.9839), confermandosi tuttavia come il modello nettamente più efficiente della suite. Con tempi di addestramento circa tre volte inferiori rispetto a GraphSAGE, TransE rappresenta il miglior compromesso (*trade-off*) tra qualità predittiva e costo computazionale per l'applicazione su reti in scala industriale.

6.2.2 Scalabilità e Overhead Infrastrutturale

Il *benchmark* infrastrutturale ha dimostrato che la scalabilità orizzontale in fase di estrazione sequenziale *batch* non produce benefici in termini di tempo di esecuzione. Le misurazioni sulle configurazioni a 1, 2 e 4 nodi si sono mantenute costanti, con oscillazioni massime nell'ordine di 1.7 secondi. Questo comportamento certifica che il vero collo di bottiglia (*bottleneck*) dell'operazione risiede nell'I/O di rete e nel coordinamento della transazione da parte del *router* del cluster, non nella potenza di calcolo disponibile.

Tuttavia, il reale valore aggiunto dell’architettura a 4 nodi si manifesta sul piano dell’affidabilità (*fault tolerance*). La ridondanza dei dati sui nodi *secondary* ha garantito la continuità operativa del database.

6.3 Contributi del lavoro

La presente tesi apporta tre contributi tecnico-scientifici distinti:

- **Validazione empirica di algoritmi su dominio reale:** Un’analisi rigorosa (su 5 cicli di validazione indipendente) del comportamento di GraphSAGE, TransE e HashGNN su un task biomedico, offrendo linee guida applicative basate sul *trade-off* tra accuratezza e tempi di calcolo.
- **Ingegnerizzazione infrastrutturale *Enterprise*:** Il rilascio di un’infrastruttura completa e versionata (*Infrastructure as Code* tramite Ansible) per il *deploy*, lo spegnimento chirurgico e il monitoraggio di un cluster distribuito Neo4j dedicato all’alimentazione di flussi di Machine Learning.
- **Profilazione dell’I/O su grafi:** La quantificazione misurabile dell’overhead di rete su database a grafo distribuiti durante carichi di estrazione sequenziali, sfatando il mito della scalabilità lineare *out-of-the-box* per ogni tipologia di interrogazione.

6.4 Limitazioni e Sviluppi Futuri

Nonostante i risultati ottenuti, l’analisi presenta limitazioni che offrono interessanti spunti per espansioni future.

In primo luogo, l’addestramento si è concentrato esclusivamente sulla topologia Chemical→Disease. Un’estensione naturale prevede l’inclusione di ulteriori entità biologiche (es. *Gene*, *Protein*) e tipi di arco, scenario in cui i modelli puramente eterogenei come HashGNN o modelli basati su meta-path (es. HAN) [10] potrebbero esprimere il loro pieno potenziale.

In aggiunta, la sperimentazione di HashGNN su un numero significativamente superiore di iterazioni (300-500 epoche) potrebbe valutare le reali dinamiche di convergenza a lungo termine del modello.

Sul fronte dell'ingegneria dei dati, il *benchmark* limitato alle query sequenziali invita a una profilazione più aggressiva tramite interrogazioni parallele asincrone generate da molteplici *client*, condizione ideale per saturare le risorse di un cluster a 4 nodi e misurarne i reali vantaggi in lettura. Un paradigma architetturale speculare consisterebbe nell'eliminare totalmente la latenza di estrazione demandando l'addestramento alla libreria *Graph Data Science* (GDS) interna a Neo4j, eseguendo le elaborazioni vettoriali direttamente sulle istanze del database.

Infine, la concretizzazione ultima di questa pipeline risiede nel suo utilizzo farmaceutico attivo: analizzare le code della distribuzione delle probabilità per identificare i collegamenti Chemical→Disease altamente probabili ma inesistenti nel database attuale, proponendoli alla comunità medica come candidati primari per validazioni *in vitro* e veri e propri *drug repurposing*.

Bibliografia

- [1] Cassandra Königs, Marcel Friedrichs, and Theresa Dietrich. The heterogeneous pharmacological medical biochemical network pharnebnet. *Scientific Data*, 9(1):393, 2022.
- [2] Cassandra Königs, Marcel Friedrichs, and Theresa Dietrich. The heterogeneous pharmacological medical biochemical network PharMeBInet. *Scientific Data*, 9(1):393, 2022.
- [3] Paolo Atzeni, Stefano Ceri, Stefano Paraboschi, and Riccardo Torlone. *Basi di Dati*. McGraw-Hill Education, 5 edition, 2018.
- [4] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, 2013.
- [5] Will Hamilton, Zhitaoy Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [6] Neo4j Inc. HashGNN – Neo4j graph data science. <https://neo4j.com/docs/graph-data-science/current/machine-learning/node-embeddings/hashgnn/>. Accesso: Giugno 2026.

- [7] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998.
- [8] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [9] Cassandra Königs, Marcel Friedrichs, and Theresa Dietrich. PharMeBINet – the heterogeneous pharmacological medical biochemical network. <https://doi.org/10.5281/zenodo.5816976>, 2022. Accesso: Giugno 2026.
- [10] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032, 2019.